Christof Teuscher

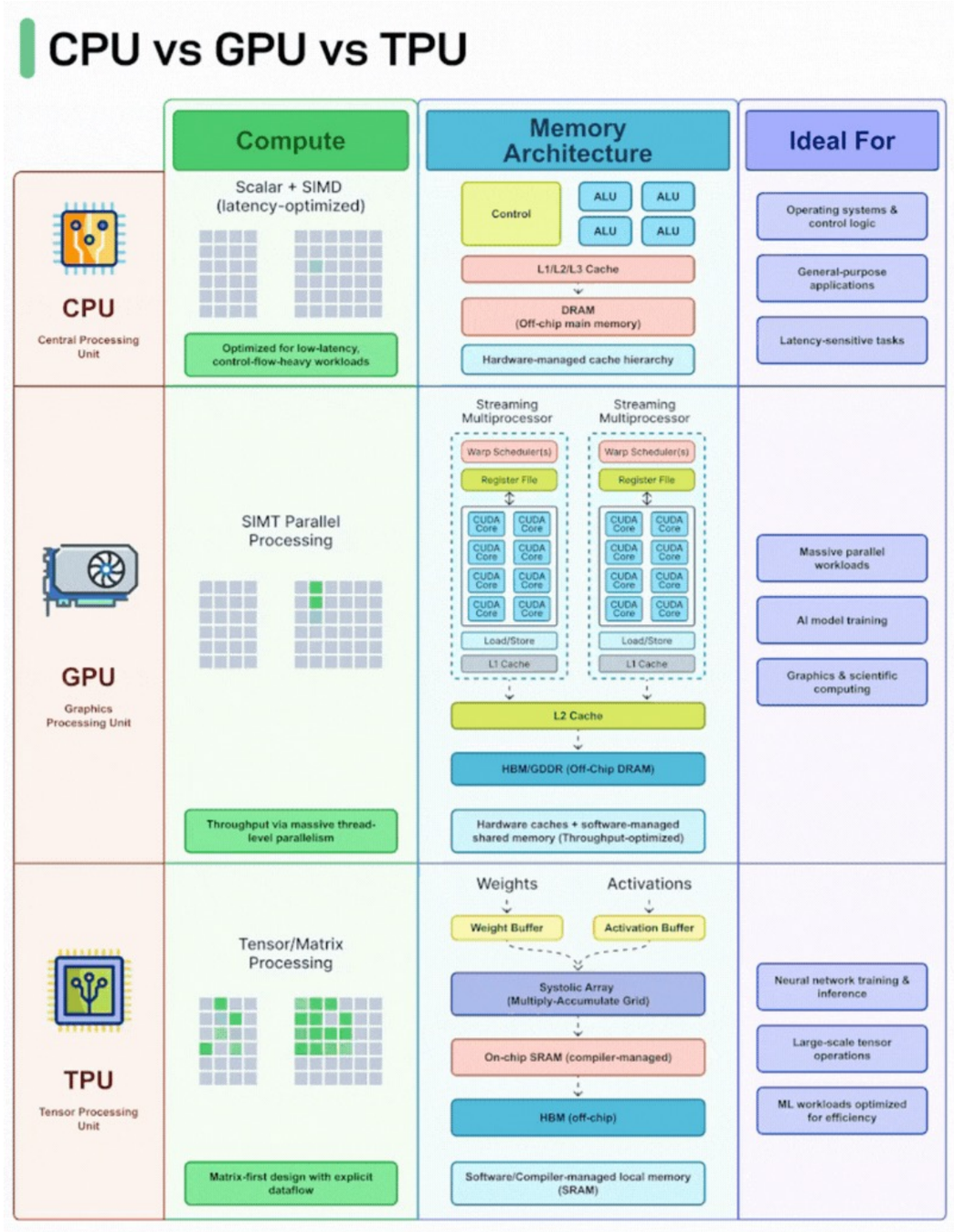
ECE 410/510: Hardware for AI and ML, 2026

TPU + CNN/DNN + Transformers

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu

teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY



Daniel Schneider • 3rd+
I find the best hardware for your (AI) needs and systems | Technic...
1d •

Connect

Use the right silicon for the right workload. GPU-only can mean leaving performance on the table. Here comes a breakdown:

A lot of people talk about compute like there is one winner. There isn't.
CPU, GPU, and TPU are fast for different reasons, because they are built for different kinds of work.

The CPU is still the best tool when the workload is messy: branching logic, system control, interrupts, orchestration, databases, and applications that need flexibility more than raw parallel throughput.

The GPU takes over when the same operation has to run across massive amounts of data. That is why it shines in graphics, matrix-heavy compute, and many AI workloads where parallelism is the real advantage.

The TPU goes even further in one direction. It is not trying to be general-purpose hardware; it is designed to push tensor and matrix operations through dedicated data paths with very high efficiency.

That is the real lesson: performance is not just about more cores, more watts, or more hype. It is about how well the architecture matches the workload.
So no, the GPU is not automatically the answer to everything.

- If your job is control-heavy, a CPU wins.
- If your job is massively parallel, a GPU wins.

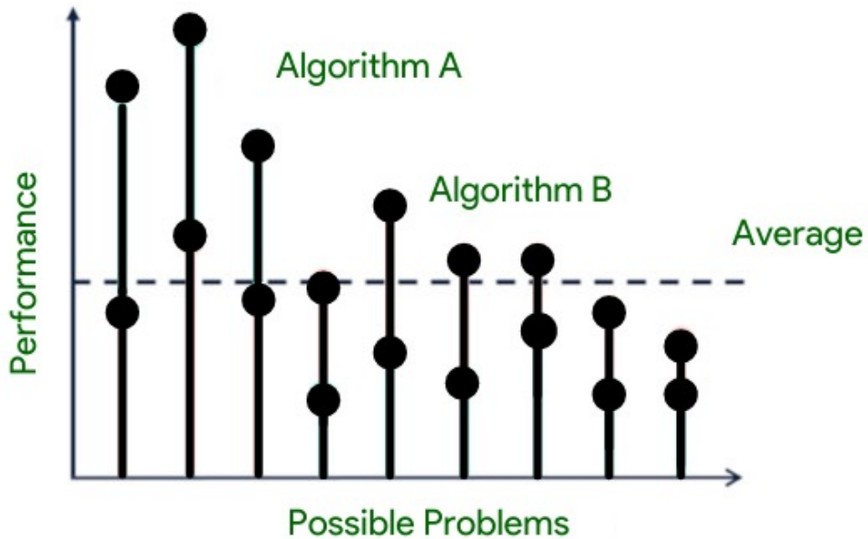
For sure a LLM is per defintion a batch of many parallel tasks but the question is if you run it 100x in a row or parallel or just once. If just once or not that often in short time - you might not need a super highend GPU.

If your job maps cleanly to tensor pipelines, specialized hardware like a TPU can leave both behind.

The best system is not the one with the most compute on paper. It is the one that wastes the least effort doing the work you actually need.

No Free Lunch theorem (NFL)

- The "No Free Lunch" (NFL) theorem in optimization and machine learning states that, when averaged across all possible problems, all optimization algorithms perform equally well. I.e., **no algorithm consistently outperforms random search or any other algorithm.**
- The NFL theorem was formalized by David Wolpert and William Macready in 1997.
- **This principle has profound implications for ML and HW optimization:**
  - It explains why specialized algorithms/architectures work well for specific domains but fail in others.
  - It highlights the importance of incorporating domain knowledge when selecting algorithms/architectures.
  - It reminds us that claims about algorithm/architecture superiority must specify the context/problem class.



https://www.geeksforgeeks.org/what-is-no-free-lunch-theorem

https://ieeexplore.ieee.org/document/585893

Discovering a Periodic Table for Machine Learning

"All these algorithms learn a **specific kind of relationship between data points**. While each algorithm may accomplish that in a slightly different way, the core mathematics behind each approach is the same."

$$\mathcal{L} = \int KL\big(p(j \mid i) \,||\, q(j \mid i)\big)\, di$$

Generalized loss function for all algorithms

Kullback-Leibler (KL) divergence, also known as relative entropy, **measures the difference between two probability distributions**. It quantifies how much information is lost if one distribution is used to approximate another

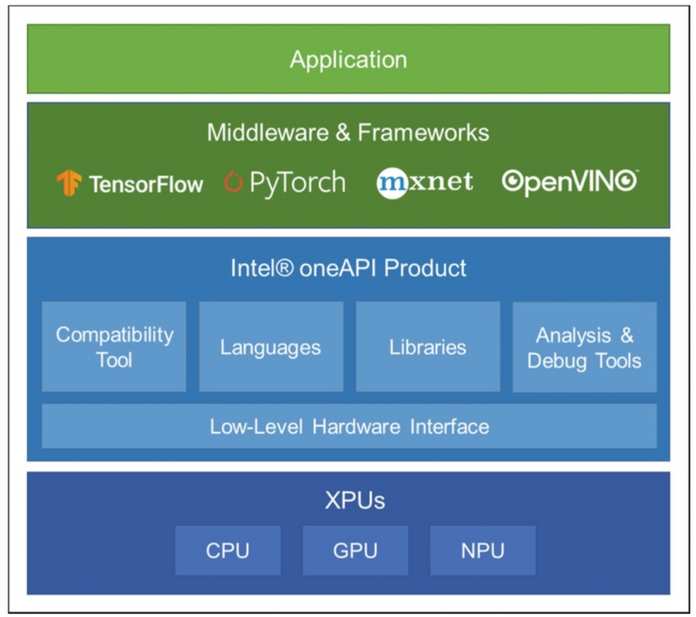
Learned Representation \ Supervisory Signal

	Gaussian	Student-T	Identity	Graph Kernel Weights	Uniform over K-Neighbors	Uniform over Positive Pairs	Cross-Modal Pairs	Uniform over Classes	Data-Label Pairs
Gaussian	SNE [Hinton 2002] X-Sample CL [Sobal 2025]	Dual t-SNE		SNE Graph Embeddings	SNE with Uniform Affinities LGSimCLR [El Banani 2023]	InfoNCE [Bachman 2019] SimCLR [Chen 2020] MoCoV3 [Chen 2021]	CLIP [Radford 2021] CMC [Tian 2020]	SupCon [Khosla 2020]	Cross Entropy [Good 1963]
Gaussian σ→∞			PCA [Pearson 1901]			VI-Reg [Bardes 2021]	Average Margin CLIP	Average Margin SupCon	
Gaussian σ→0						Triplet Loss [Schultz 2004]	Triplet CLIP	Triplet SupCon	Error rate
Student-T	t-SNE [Van der Maaten 2008]	Doubly t-SNE		t-SNE Graph Embedding	t-SNE with Uniform Affinities	t-SimCNE [Böhm 2023] t-SimCLR [Hu 2023]	t-CLIP	t-SupCon	Harmonic Loss [Baek 2025]
Cluster Probabilities	K-Means [Macqueen 1967]	t K-Means		Normalized Cuts [Shi 2000]	DCD [Yang 2012]	InfoNCE Clustering [Ours]		Supervised Clustering	

Dimensionality Reduction | Cluster Learning | Unimodal SSL | Multimodal SSL | Supervised Learning | Interpretation of Gaps

Deep learning frameworks

1. Deep learning frameworks provide interfaces to implement AI functionalities regardless of hardware or computing platforms.
2. Most deep learning frameworks also optimize the network in a **hardware-friendly manner** and utilize the hardware resources and acceleration functions as much as possible.
3. Because GPUs are commonly used in deep learning, deep learning frameworks **widely support constructing and deploying neural networks on GPU** and actively reflect the acceleration support from GPU vendors.
4. In addition, they provide optimizations on standalone accelerators such as **TPU** and specific environments such as cloud servers.



TensorFlow

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

PyTorch & CNNs

For CNNs, the mapping process involves:

1. **Tensor operations**: PyTorch represents all data (images, weights, activations) as tensors that can be moved between CPU and GPU memory.
2. **Layer-by-layer optimization**: Each CNN layer (convolution, pooling, etc.) is mapped to specific GPU kernels that efficiently perform the required operations in parallel.
3. **Memory management**: PyTorch handles the complex memory allocations and transfers between host (CPU) and device (GPU) memory.
4. **Computation graphs**: PyTorch builds a dynamic computation graph that tracks operations and enables automatic differentiation for training.
5. **Batching**: Multiple inputs are processed simultaneously to maximize GPU utilization.
6. For the **convolution operations** specifically, PyTorch typically uses highly optimized libraries like cuDNN that implement various algorithms (direct convolution, FFT-based, Winograd, etc.) and automatically select the most efficient one based on input size and available GPU resources.

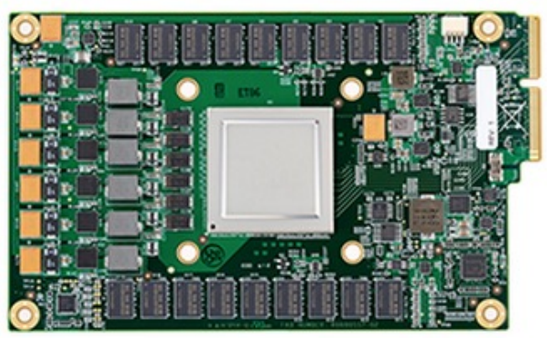
Christof Teuscher • teuscher@pdx.edu

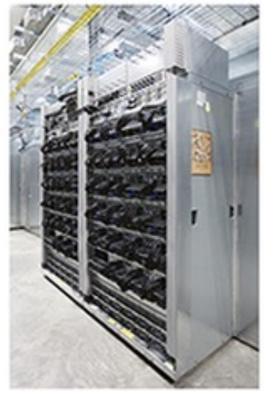
PyTorch

```python
import torch
import torch.nn as nn
import torch.optim as optim

# Define the network architecture
class SimpleMLPNet(nn.Module):
    def __init__(self):
        super(SimpleMLPNet, self).__init__()
        # First layer: 4 input neurons -> 5 hidden neurons
        self.fc1 = nn.Linear(4, 5)
        # Second layer: 5 hidden neurons -> 1 output neuron
        self.fc2 = nn.Linear(5, 1)
        # ReLU activation for hidden layer
        self.relu = nn.ReLU()

    def forward(self, x):
        # Forward pass through first layer and apply ReLU
        x = self.relu(self.fc1(x))
        # Forward pass through output layer (no activation)
        x = self.fc2(x)
        return x
```

```python
# Create a batch of random input data (16 samples, 4 features each)
batch_size = 16
input_size = 4
x = torch.rand(batch_size, input_size)

# Create the network
model = SimpleMLPNet()

# Move everything to GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
x = x.to(device)

# Forward pass
with torch.no_grad():  # No need to track gradients for inference
    output = model(x)

# Print some sample outputs
print("Sample outputs from MLP:")
for i in range(5):  # Print first 5 results
    print(f"Sample {i}: {output[i].item():.6f}")
```

GPU vs Tensor Processing Unit (TPU)





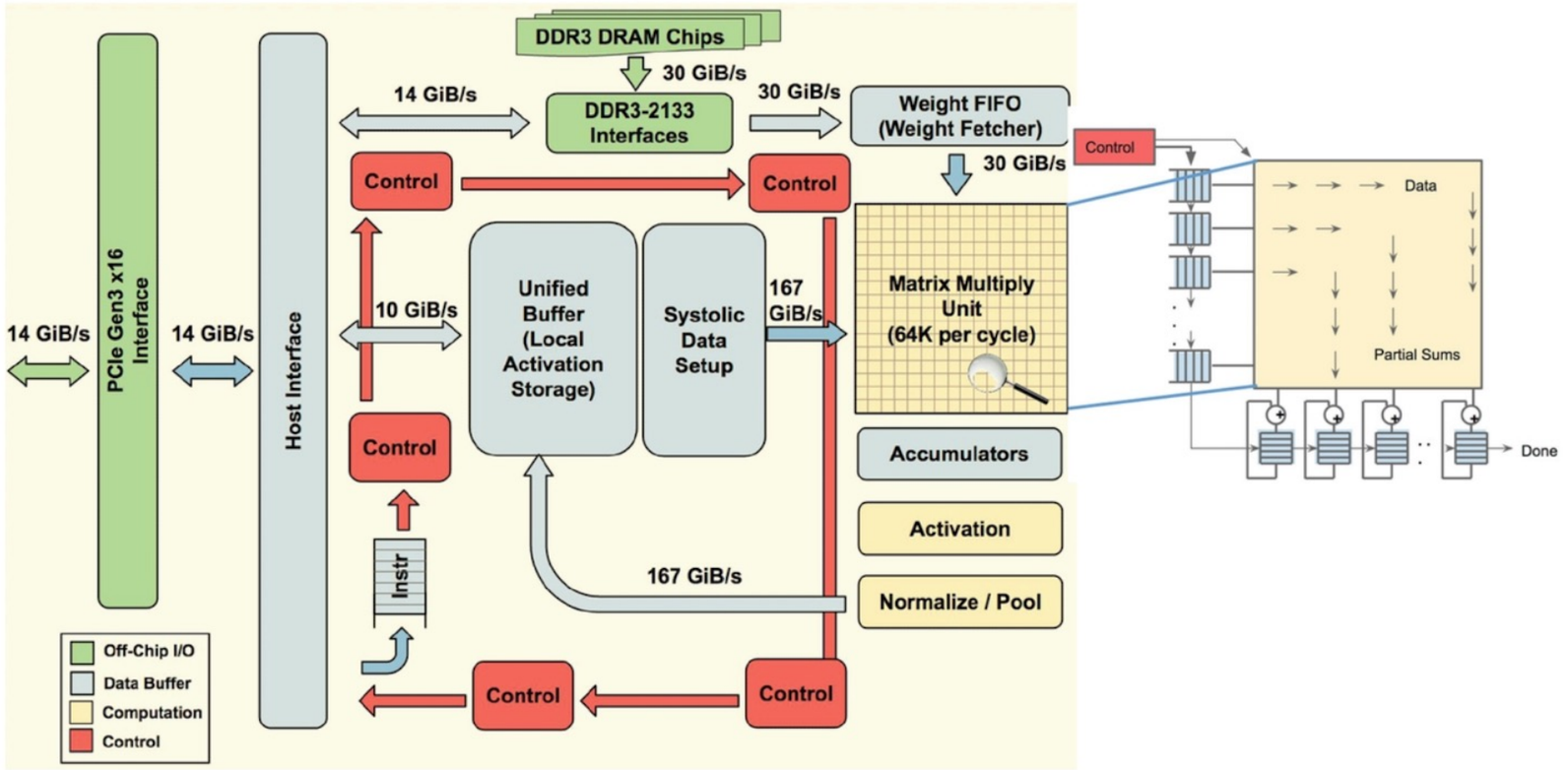
GPU vs TPU

	GPU	TPU
Origin and design purpose	Graphics	Created by Google for ML
Architecture	Thousands of smaller cores designed for parallel processing.	More specialized architecture with matrix processing units (MXUs) specifically optimized for tensor operations.
Performance focus	More flexibility, broader applicability	Optimized for matrix operations and ML workloads.
Development and availability	NVIDIA, AMD, etc.	Proprietary, only through Google cloud
Software compatibility	Various frameworks, e.g., CUDA	TensorFlow
Power efficiency	Less power-efficient, but more flexibility	More power-efficient

TPU key features

- **Matrix Multiplication Unit** (MXU): The central component designed to perform matrix operations quickly. It loads parameters and data from high-bandwidth memory (HBM), executes multiplications, and passes results to the next multiply-accumulator.
- **TensorCores**: Each TPU chip contains one or more TensorCores, with the number depending on the version. Each TensorCore consists of one or more matrix-multiply units, a vector unit, and a scalar unit.
- **Activation Unit** (AU): Provides hardwired activation functions commonly used in neural networks

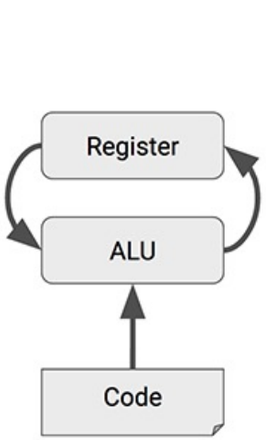
Architecture of a TPU accelerator

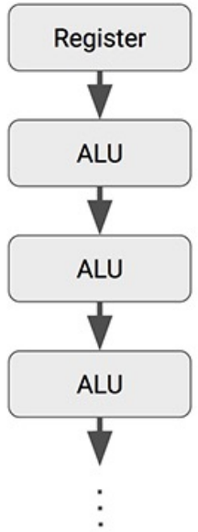


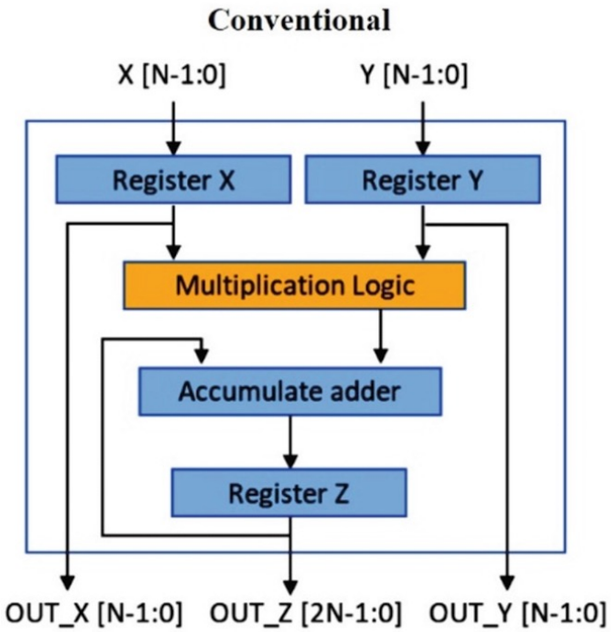
- The main component of TPU is the MXU that consists of 256 by 256 (i.e., 65,536) MACs to perform 8-bit mathematical operations.
- The MXU gets its input from the weighted FIFO and unified buffer (UB).

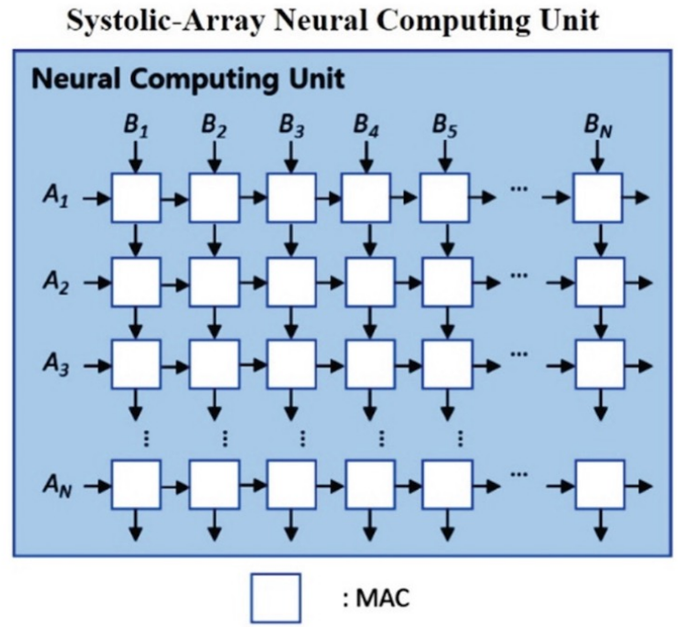
https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu

Systolic array (1)







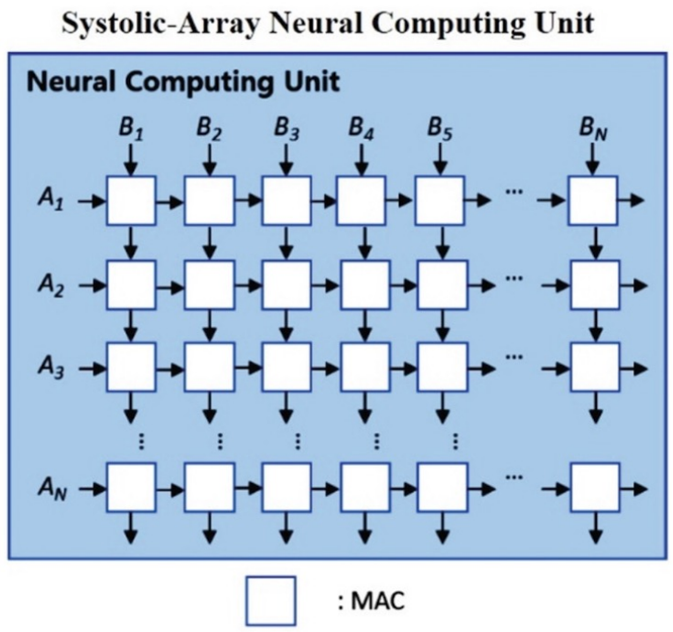


- The name "systolic" comes from an analogy to the human heart, where data pulses through the array of processors in a rhythmic, synchronized fashion.
- CPUs and GPUs often spend energy to access multiple registers per operation. A systolic array chains multiple ALUs together, reusing the result of reading a single register.

Mishra et al., 2023

Systolic array (2)

- Systolic arrays, are characterized by:
  - Pre-defined computational flow graph that connects their nodes.
  - Regular arrangement of (simple) processing elements (PEs) in a grid pattern.
  - Data flows rhythmically through the array of processors.
  - Each PE performs the same operation (typically multiply-accumulate).
  - Processing elements operate in lockstep, controlled by a global clock signal. [Wavefront computers, on the other hand, are asynchronous and data-driven.]
- In a systolic array, all processing elements execute the same instruction (or fixed function) at any given time, just on different data values.
- MISD or SIMD or ? Controversial…



Matrix multiplication

Row i →

a11	a12	a13	a14
a21	a22	a23	a24
a31	a32	a33	a34
a41	a42	a43	a44

Matrix A

×

↓ Column j

b11	b12	b13	b14
b21	b22	b23	b24
b31	b32	b33	b34
b41	b42	b43	b44

Matrix B

=

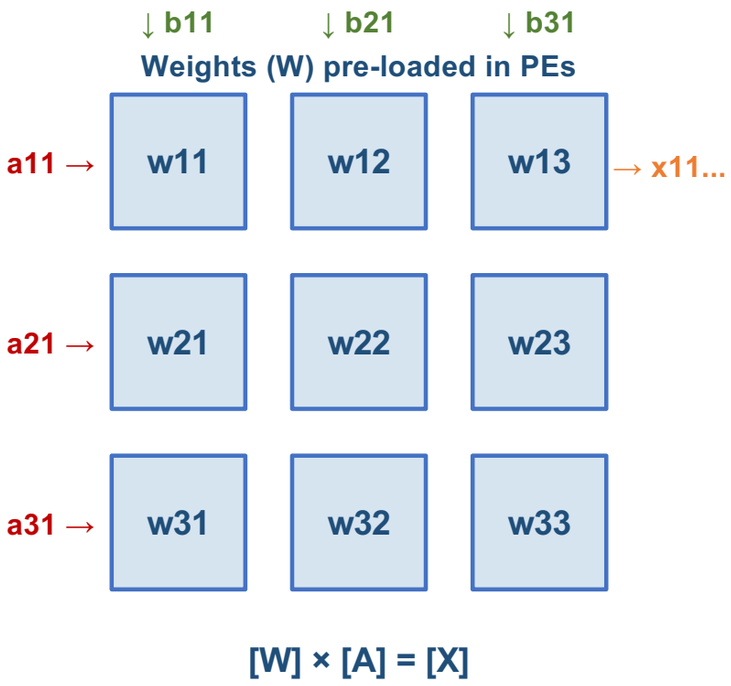
c11	c12	c13	c14
c21	c22	c23	c24
c31	c32	c33	c34
c41	c42	c43	c44

Matrix C = A × B

$c_{ij} = \sum_k a_{ik} \cdot b_{kj}$ Example: $c_{11} = a_{11}b_{11} + a_{12}b_{21} + a_{13}b_{31} + a_{14}b_{41}$

Each element of C requires N multiply-accumulate (MAC) operations. A 4×4 matrix multiply needs $4^3 = 64$ MACs total.

Systolic array: how data flows



How a systolic array computes matrix multiplication:

1. Pre-load weights
Each PE stores one weight element (wij). Weights remain stationary in the array.

2. Stream activations
Input activations (a values) flow from left to right, one element per clock cycle, staggered by row.

3. Multiply-accumulate
Each PE multiplies its weight by the incoming activation and adds to a running partial sum.

4. Pass data along
Each PE passes the activation to the right neighbor and partial sums flow downward.

5. Collect results
After N cycles of pipeline fill, one result element emerges per cycle. No wasted register reads.

Key advantage: Each weight is read once but used for many MACs, saving energy vs. GPUs/CPUs that re-read from registers each cycle.

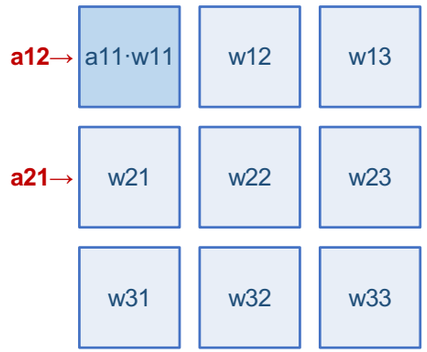
Christof Teuscher • teuscher@pdx.edu

Systolic array: 3 × 3 example step-by-step

t = 0: Data enters

a11 enters top-left PE

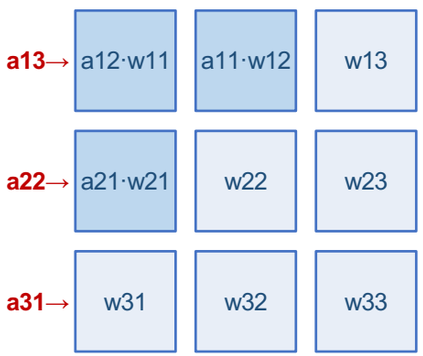
t = 1: First MACs



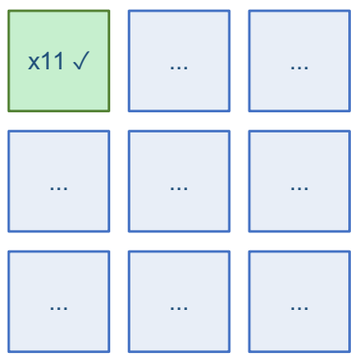
PE(0,0) computes a11·w11

t = 2: Pipeline fills

Partial sums accumulate



t = 4: First result!



x11 = a11·w11 + a12·w11 + a13·w11

- For N×N array: pipeline fill takes **2N-1 cycles**, then one result per cycle.
- Total: **3N-2 cycles for full N×N multiply**.

Christof Teuscher • teuscher@pdx.edu

Systolic Array Dataflows

Weight Stationary

- Weights stay fixed in each PE
- Activations flow left → right
- Partial sums flow top → bottom
- Minimizes weight reads from memory
- Best for: layers reused across many inputs (convolutions, large batches)

Example: Google TPU v1–v4
Weights loaded once, reused across entire input batch

Output Stationary

- Partial sums (outputs) stay in PE
- Weights stream through PEs
- Activations also stream through
- Minimizes accumulator writes
- Best for: small filters with many output pixels.

Example: ShiDianNao (vision processor)
Each PE accumulates one output element to completion

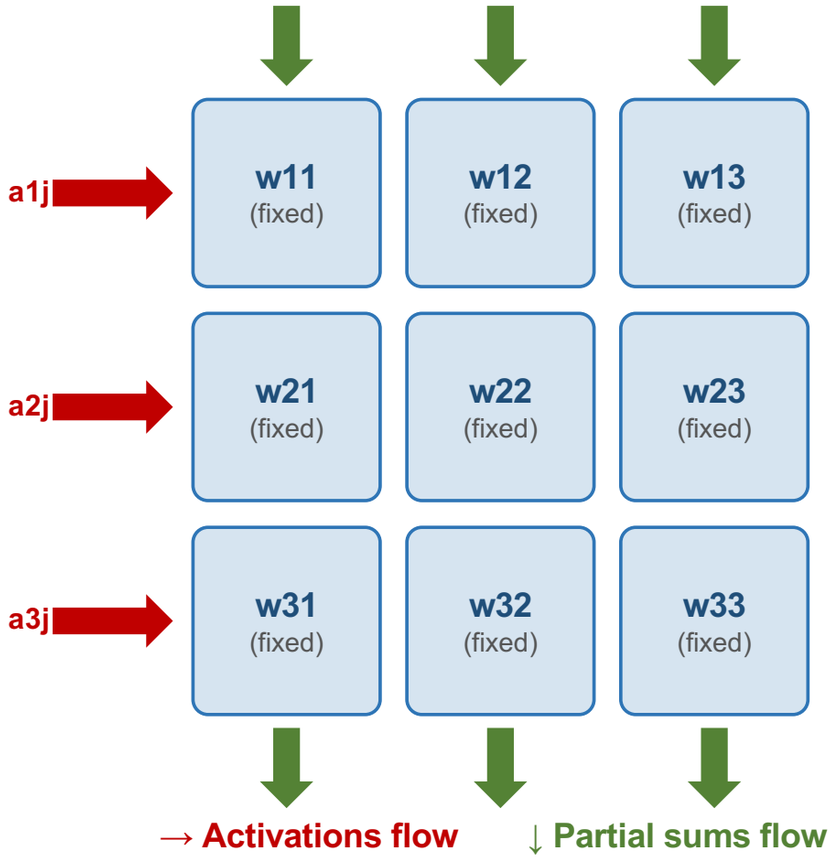
Row Stationary

- 1D row of a filter stays in PE
- Activations reused across PEs in same column
- Partial sums accumulated locally
- Maximizes reuse of ALL data types
- Best for: diverse layer shapes (CNN, FC, LSTM)

Example: MIT Eyeriss (CNN accelerator)
Optimizes energy by minimizing total data movement

Key trade-off: Each dataflow minimizes movement of one data type at the cost of moving others more. The optimal choice depends on the layer shape and batch size.

Weight-stationary dataflow (Google TPU)



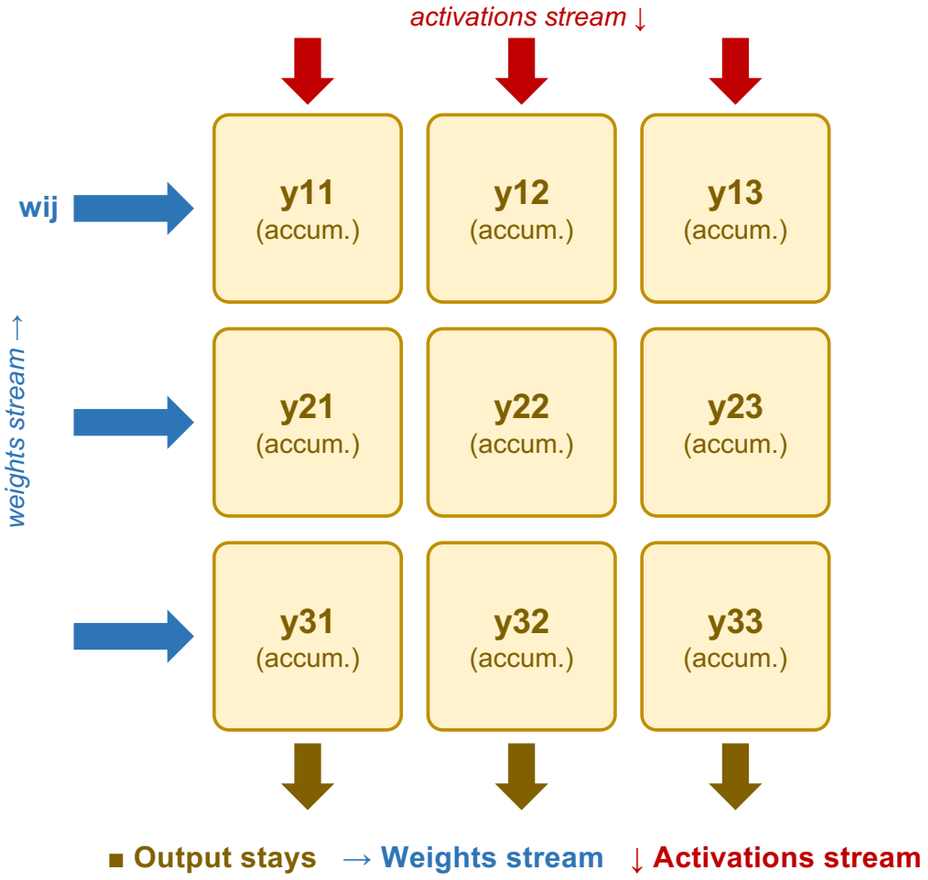
How it works

1. Weights are pre-loaded into each PE and remain fixed for the entire computation
2. Activations stream in from the left, each PE multiplies by its stored weight and passes the activation to the next PE
3. Partial sums accumulate downward: each PE adds its product to the running sum from above
4. Final outputs emerge at the bottom of each column

Why this works well for TPUs

- Same weights reused for every input in the batch → amortizes the cost of loading weights from HBM
- 256×256 MXU = 65,536 MACs per cycle, all sharing the same weight matrix
- Only N^2 local registers needed (no global register file)

Output-stationary dataflow (ShiDianNao)



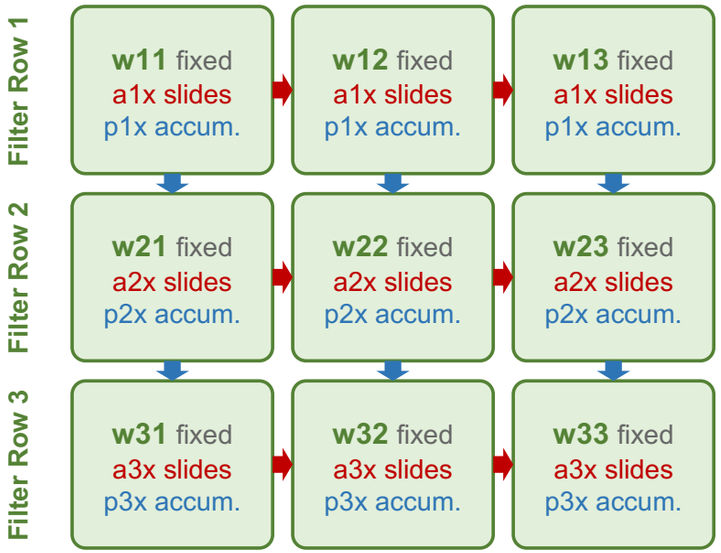
How it works

1. Each PE holds one output element (y_ij) for the entire computation
2. Weights stream in from the left (one new weight per cycle)
3. Activations stream in from the top (one new activation per cycle)
4. Each PE accumulates: y_ij += w_ik × a_kj over K cycles

Key properties

- Minimizes accumulator writes (output stays in local register)
- Both weights and activations must be fetched from memory every cycle → higher bandwidth demand
- Used by: ShiDianNao (ISCA 2015)
- Best when output matrix is small relative to input dimensions

Row-stationary dataflow (MIT Eyeriss)



■ Weight stays → Activations slide ↓ Psums accumulate

How it works

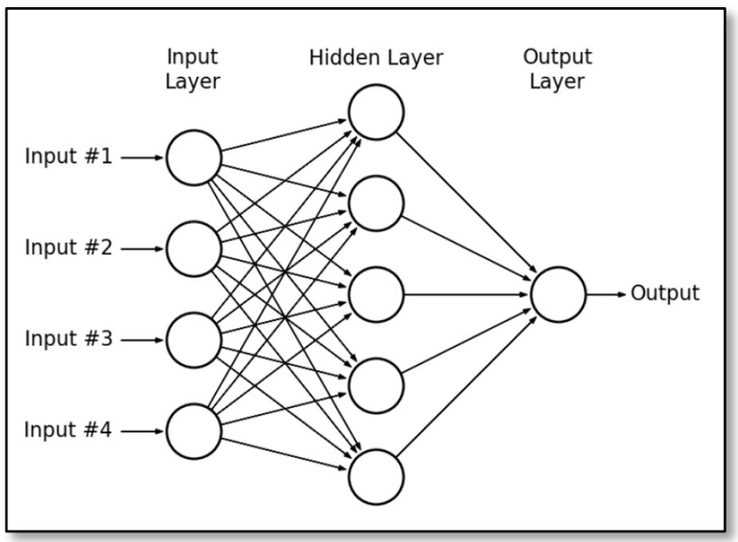
1. Each PE row processes one row of the convolution filter
2. Filter weights stay fixed in each PE (like weight-stationary)
3. Activations (input feature map rows) slide horizontally through PEs within the same row
4. Partial sums pass vertically between rows and accumulate across filter rows

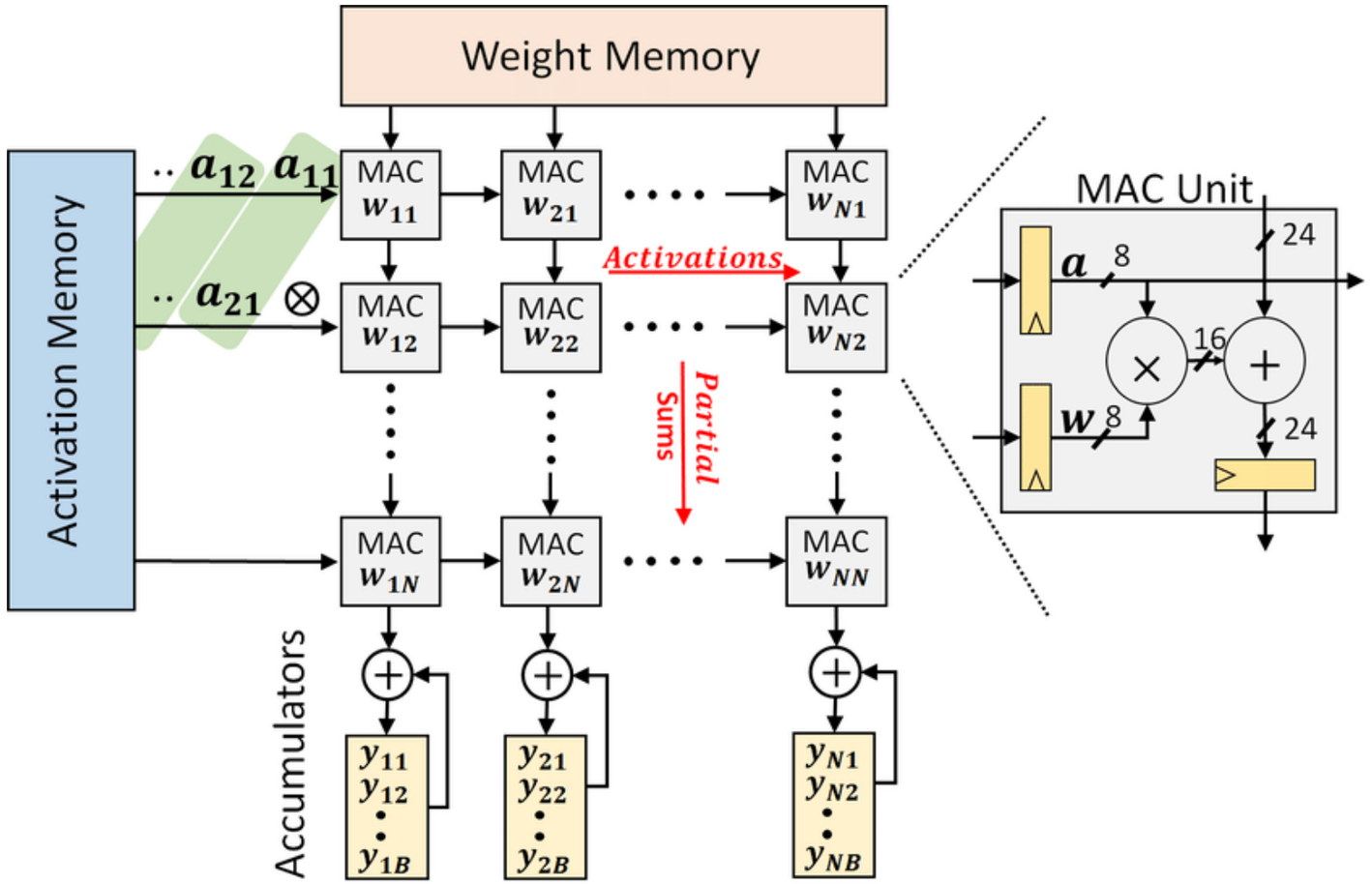
Why this is energy-optimal

- Maximizes reuse of all three data types (weights, activations, psums) at the PE level
- Adjacent PEs in a row share the same ifmap row → convolutional reuse via sliding window
- Eyeriss (MIT, 2016): 168 PEs, 16-bit fixed point, achieved 10× energy savings vs. conventional accelerators
- Adapts well to diverse layer shapes — no single data type dominates the design

input feature map = ifmap = 1D row
One ifmap row stays associated with one PE row

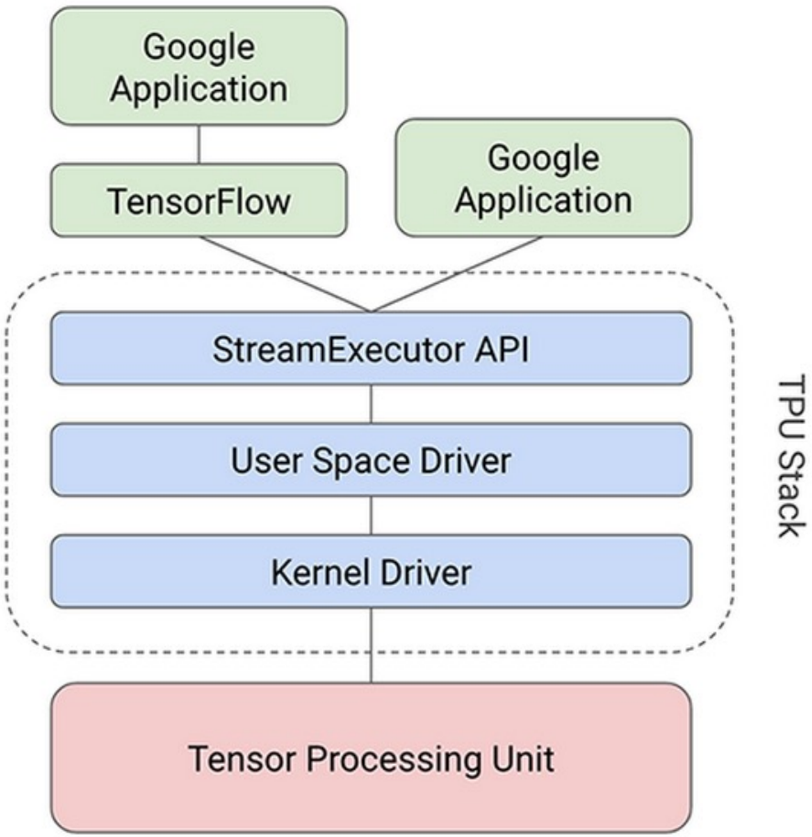
How to map a deep neural net on a systolic array





https://doi.org/10.1145/3195970.3196129

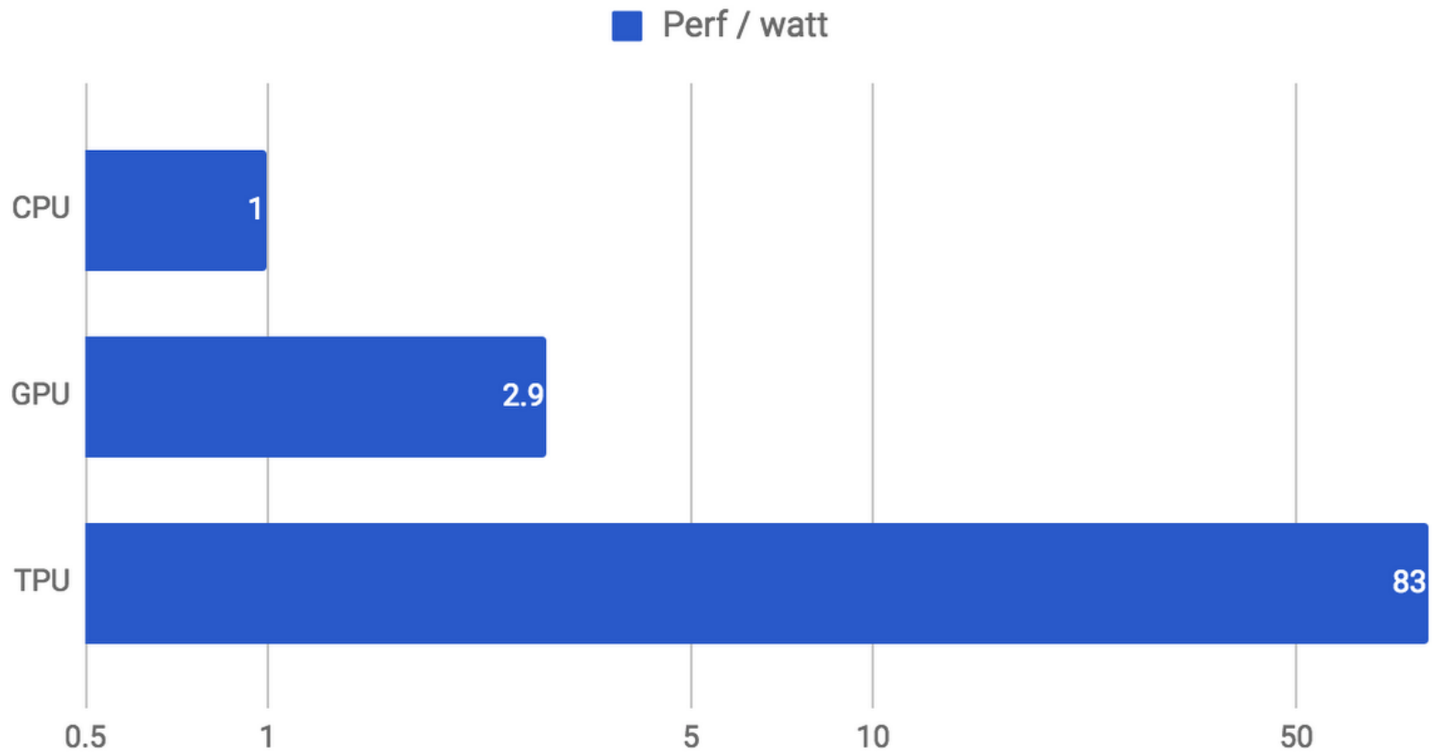
Google's TPU



	Operations per cycle
CPU	a few
CPU (vector extension)	tens
GPU	tens of thousands
TPU	hundreds of thousands, up to 128K

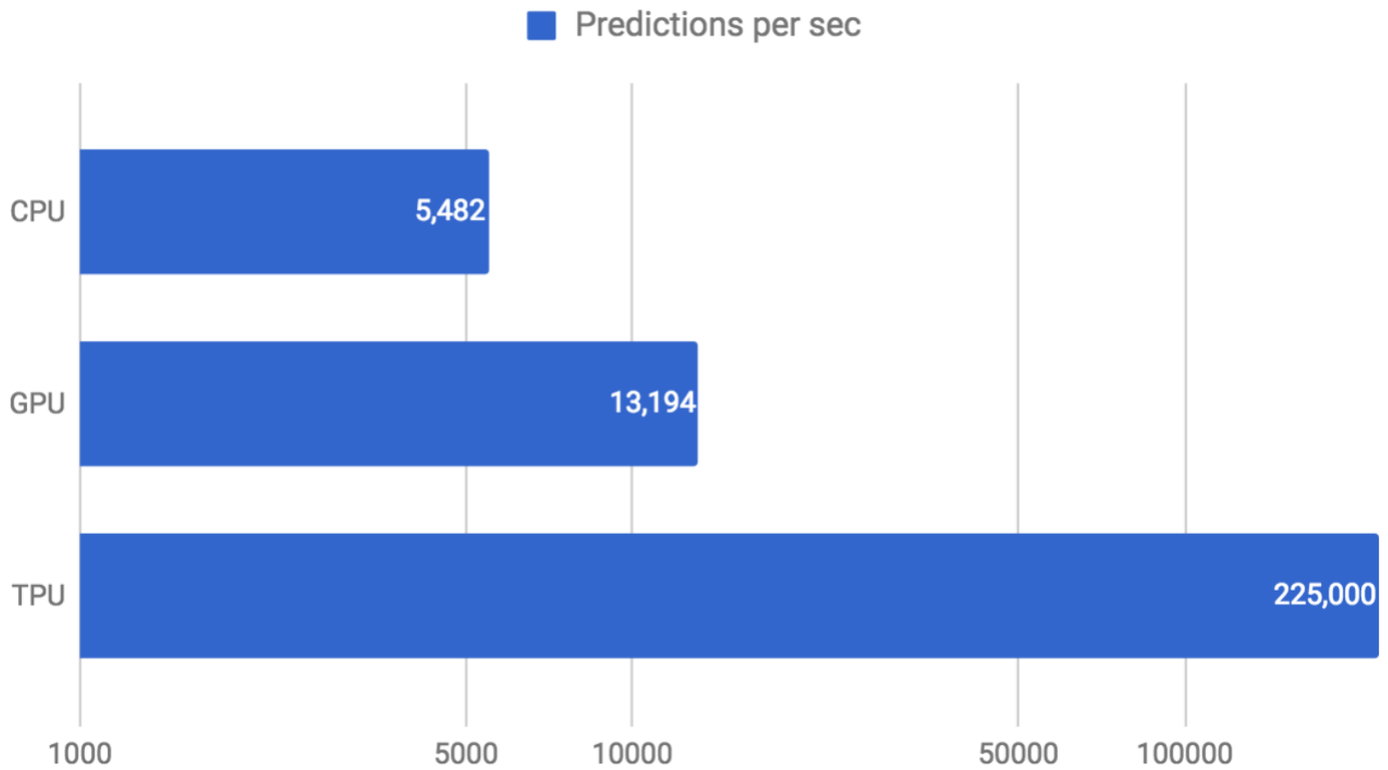
https://cloud.google.com/tpu/docs/system-architecture-tpu-vm

Google's TPU



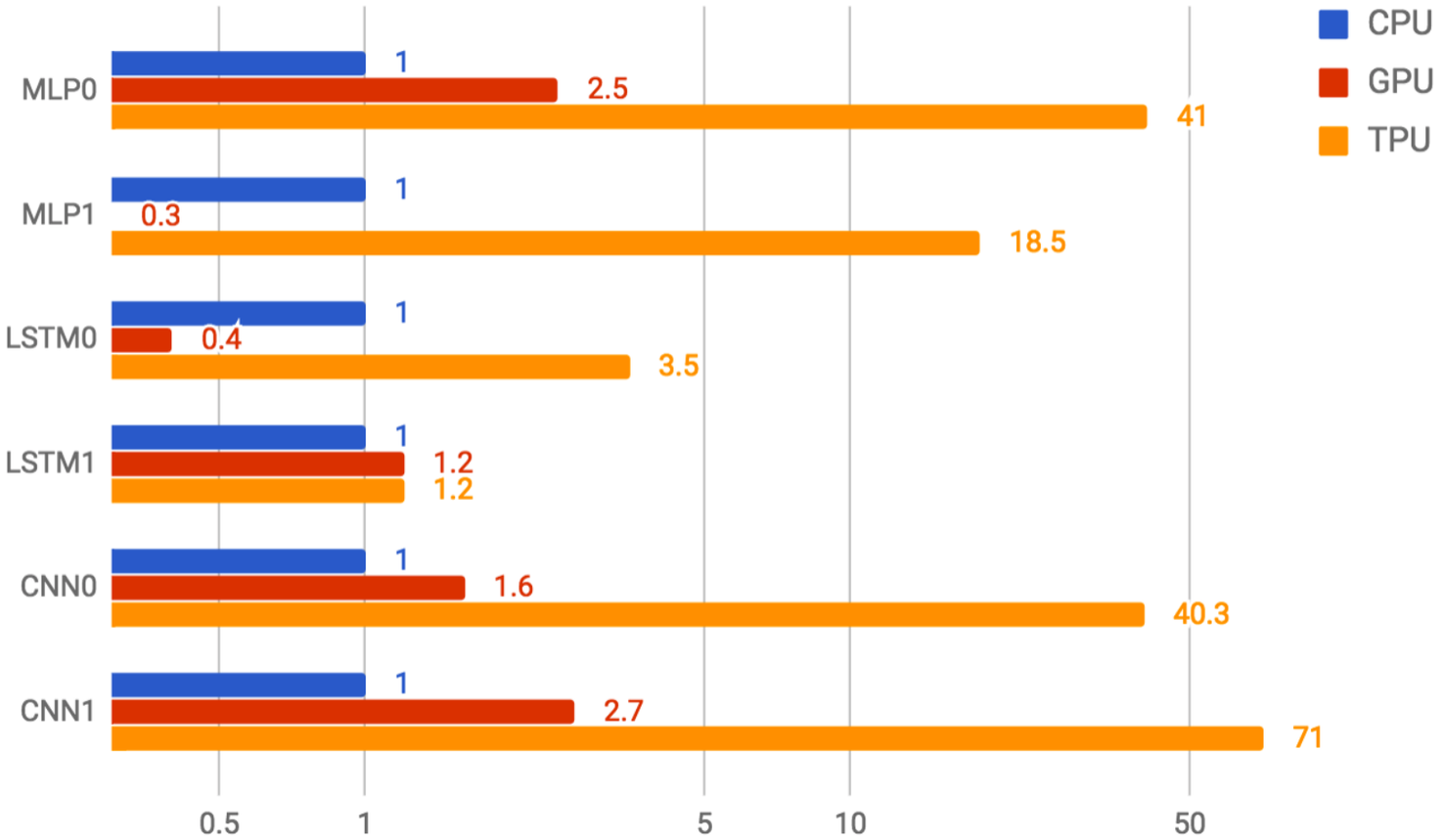
https://cloud.google.com/tpu/docs/system-architecture-tpu-vm

Google's TPU



https://cloud.google.com/tpu/docs/system-architecture-tpu-vm

Google's TPU



https://cloud.google.com/tpu/docs/system-architecture-tpu-vm

Christof Teuscher • teuscher@pdx.edu

TPU generation roadmap: v4 → v7 (Ironwood)

Spec	TPU v4	TPU v5e	TPU v5p	v6e (Trillium)	v7 (Ironwood)
Year	2022	2023	2023	2024	2025
MXU Size	128×128	128×128	128×128	256×256	256×256
Peak BF16	275 TFLOPS	197 TFLOPS	459 TFLOPS	~918 TFLOPS	~2,300 TFLOPS
HBM	32 GB	16 GB	95 GB	32 GB	192 GB
HBM BW	1,200 GB/s	819 GB/s	2,765 GB/s	1,600 GB/s	7,400 GB/s
Topology	3D torus	2D torus	3D torus	2D torus	2D torus
Max chips/pod	4,096	256	8,960	256	9,216
FP8 Native	No	No	No	No	Yes (FP8)

Sources: cloud.google.com/tpu; Introl TPU Guide (2025)

Industry-wide shift toward low-precision

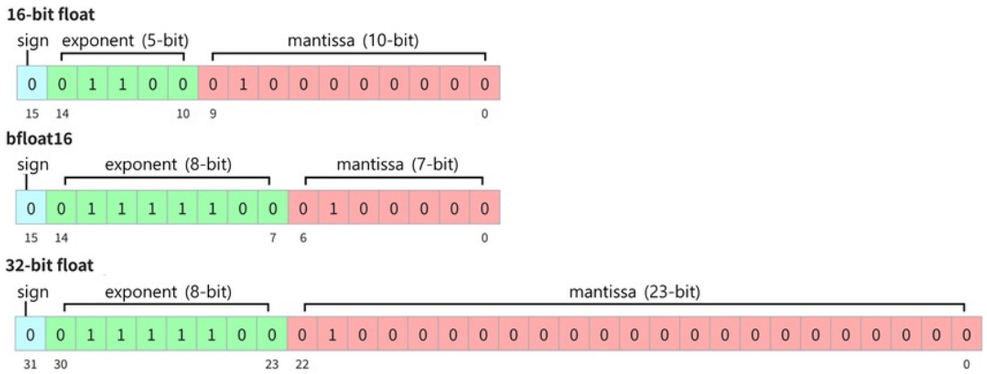
Google explicitly differentiates efficiency-optimized (v5e, v6e) from performance-optimized (v5p, v7) variants, letting customers choose the cost/performance tradeoff. The "e" chips trade raw FLOPS for better FLOPS-per-dollar for large-scale inference.

BF16 (1)

- BF16 (Brain Float 16) is a 16-bit floating-point format developed by Google Brain specifically for machine learning.
- BF16 keeps the full 8-bit exponent from FP32, which means it has the same dynamic range as FP32
- You can represent the same magnitude of very large and very small numbers. It just sacrifices precision (fewer mantissa bits).

BF16 (2)

- Neural network training is **sensitive to dynamic range** (gradients can span many orders of magnitude) but relatively tolerant of precision loss, i.e., a weight being 0.1234 vs 0.1237 rarely matters.
- **FP16 has a much smaller exponent range and frequently causes overflow/underflow during training, requiring loss scaling workarounds.**
- BF16 avoids those instabilities entirely, making it a drop-in replacement for FP32 in most training workloads.
- Converting between BF16 and FP32 is trivial: **just add or drop the last 16 mantissa bits, no reformatting needed.**



Transformers

2017 landmark paper

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

https://arxiv.org/pdf/1706.03762

2017 landmark paper

Attention is all you need

A Vaswani, N Shazeer, N Parmar… - Advances in neural …, 2017 - proceedings.neurips.cc

… to attend to **all** positions in the decoder up to and including that position. **We need** to prevent … **We** implement this inside of scaled dot-product **attention** by masking out (setting to −∞) …

☆ Save 99 Cite Cited by 243921 Related articles All 71 versions »

A mathematical theory of communication

CE Shannon - The Bell system technical journal, 1948 - ieeexplore.ieee.org

… By a **communication** system we will mean a system of the type … as **mathematical** entities, suitably idealized from their physical counterparts. We may roughly classify **communication** …

☆ Save 99 Cite Cited by 109860 Related articles All 85 versions Web of Science: 19192 »

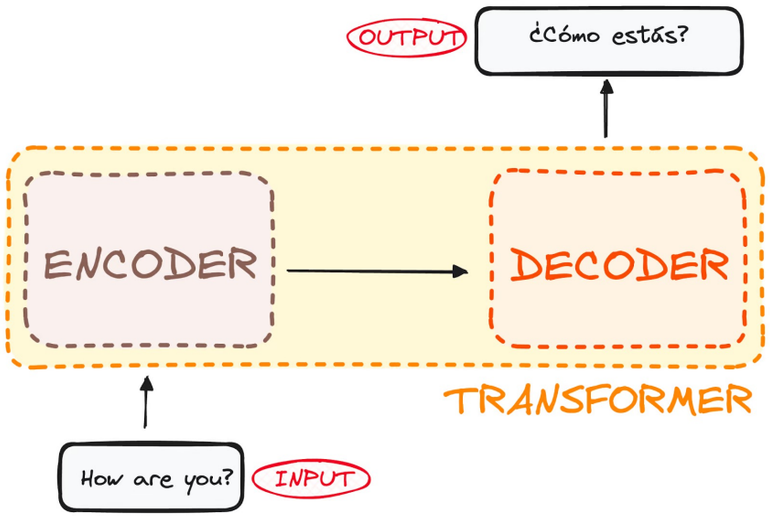
https://arxiv.org/pdf/1706.03762

What is a transformer?

A transformer is designed to comprehend context and meaning by analyzing the relationship between different elements.

The model is primarily composed of two blocks:

- **Encoder** (left): The encoder receives an input and builds a representation of it (its features). This means that the model is optimized to **acquire understanding** from the input.
- **Decoder** (right): The decoder uses the encoder's representation (features) along with other inputs to generate a target sequence. This means that the model is optimized for **generating outputs**.

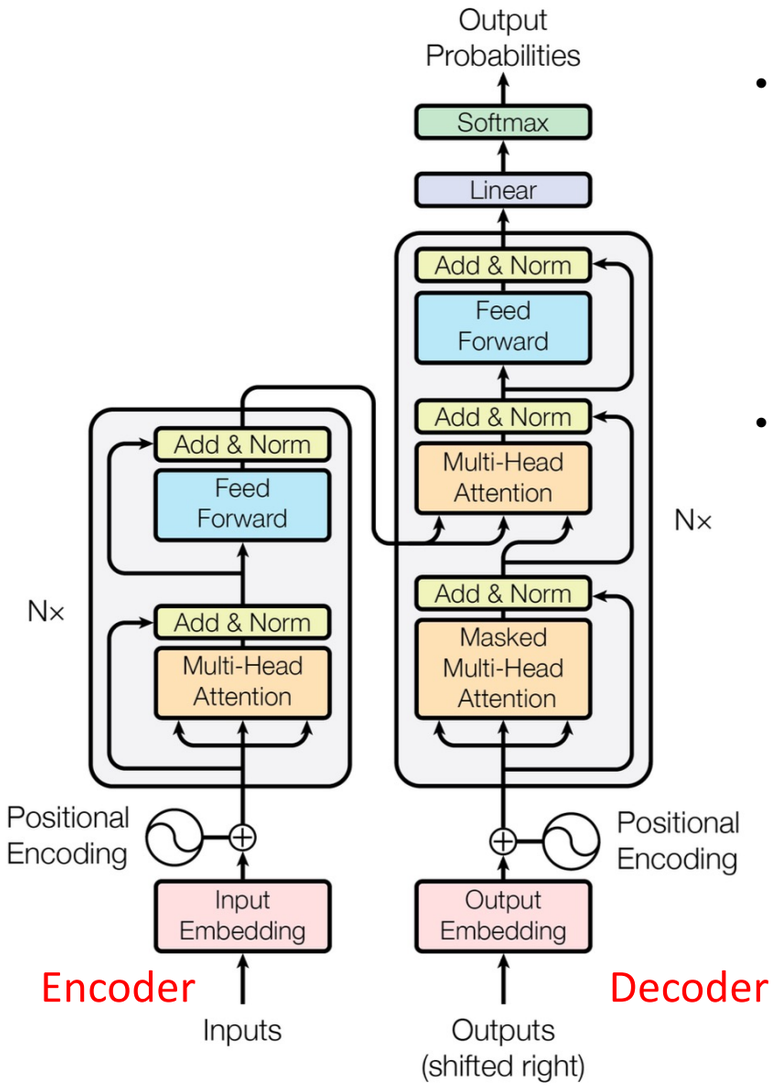


https://huggingface.co/learn/llm-course/en/chapter1/4
https://www.datacamp.com/tutorial/how-transformers-work

What is a transformer?

"A transformer model is a neural network that learns context and thus meaning by **tracking relationships in sequential data** like the words in this sentence." **Transformers made self-supervised learning possible.**

https://arxiv.org/pdf/1706.03762

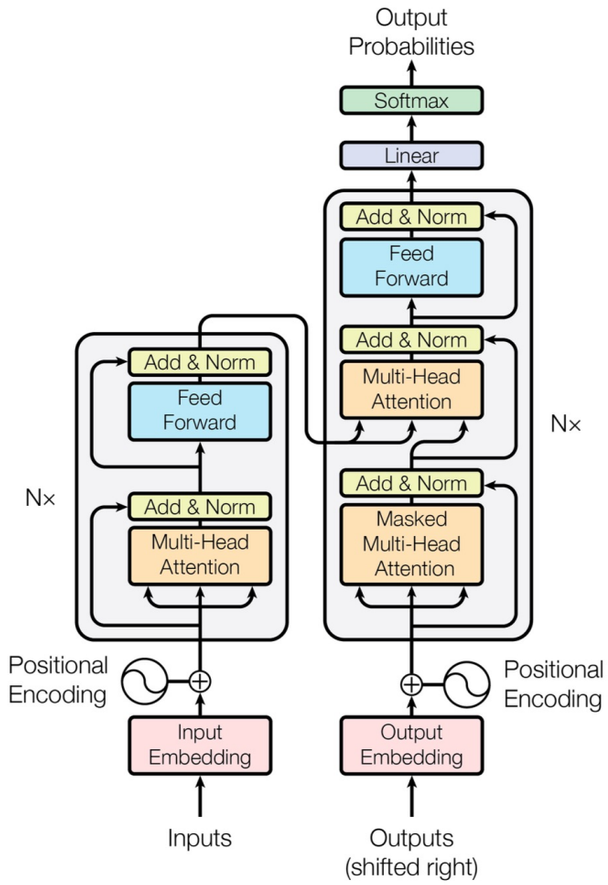


- "Transformer models apply an evolving set of mathematical techniques, called **attention** or **self-attention**, to detect subtle ways even distant data elements in a series influence and depend on each other."
- Self-attention is asking, for every token: **"which other tokens in this sequence are most relevant to understanding me?"** It does this by learning **three projections** for each token, i.e., Query (Q), Key (K), and Value (V):
  - Q: "What am I looking for?"
  - K: "What do I offer?"
  - V: "What do I actually contribute if selected?"

teuscher Lab
teuscher-lab.com

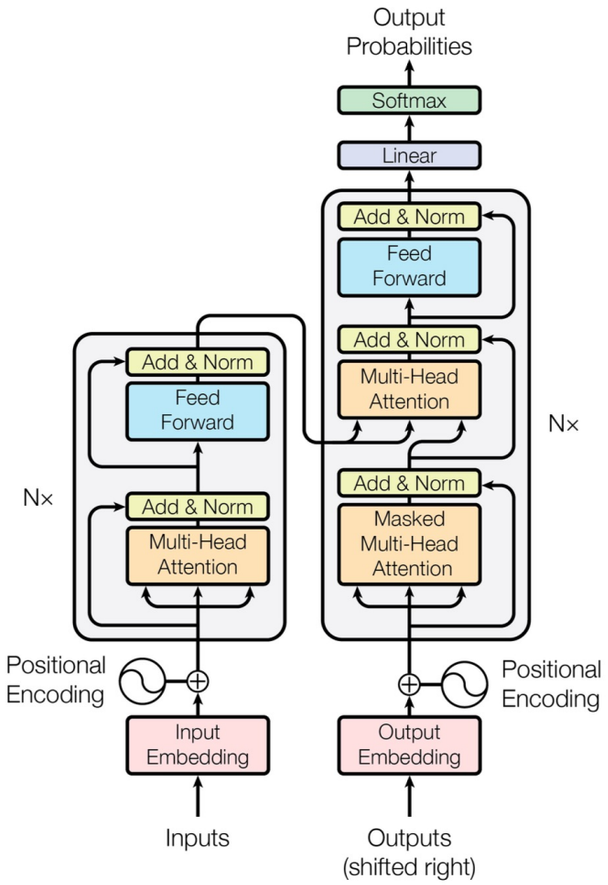
Portland State UNIVERSITY

What is recurrent in a transformer?



https://arxiv.org/pdf/1706.03762
https://blogs.nvidia.com/blog/what-is-a-transformer-model

What is recurrent in a transformer?



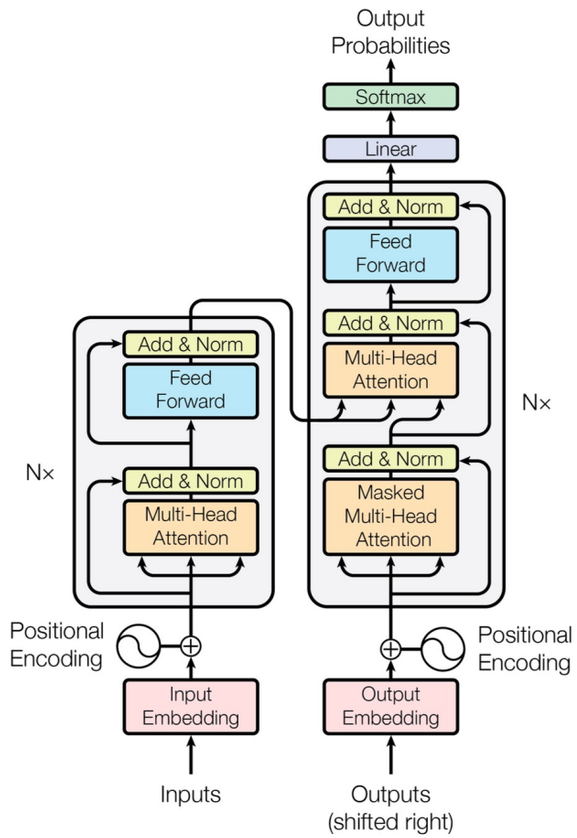
- **Transformers are explicitly non-recurrent.**
- The "Attention Is All You Need" paper (Vaswani et al., 2017) was a direct response to RNNs/LSTMs, which are recurrent. RNNs/LSTMs process tokens one at a time, passing a hidden state from step to step. That sequential dependency is what makes RNNs slow to train (can't parallelize across time steps) and bad at long-range dependencies (the hidden state bottleneck).

- **The Transformer replaces recurrence entirely with self-attention, which**:
  - Processes all tokens simultaneously in parallel
  - Connects every token directly to every other token in a single operation, i.e., no hidden state to propagate
  - Has no notion of "previous step," positional encodings handle order instead (time is replaced by position).

https://arxiv.org/pdf/1706.03762
https://blogs.nvidia.com/blog/what-is-a-transformer-model

What is a transformer?

- Addresses all drawbacks of recurrent neural nets (RNN)
- No labels (costly and time-consuming)
- Very parallelizable
- Higher performance
- Like most neural networks, transformer models are basically large **encoder/decoder (compression/decompression)** blocks that process data.
- Transformers use **positional encoders** to tag data elements coming in and out of the network.
- **Attention units** follow these tags, calculating a kind of algebraic map of how each element relates to the others.
- **Attention queries** are typically executed in parallel by calculating a matrix of equations in what's called **multi-headed attention**.



https://arxiv.org/pdf/1706.03762
https://blogs.nvidia.com/blog/what-is-a-transformer-model

Christof Teuscher • teuscher@pdx.edu

What is a transformer?

A transformer is designed to comprehend context and meaning by analyzing the relationship between different elements.

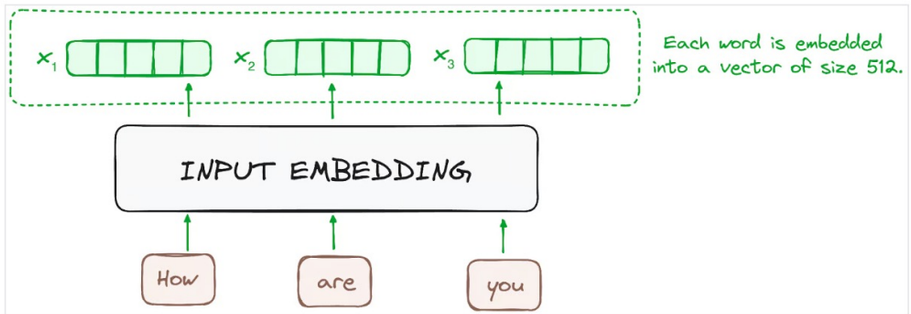


Image by the author. Encoder's workflow. Input embedding.

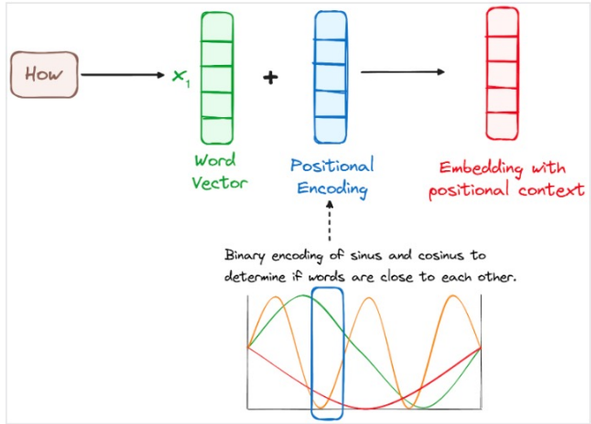


Image by the author. Encoder's workflow. Positional encoding.

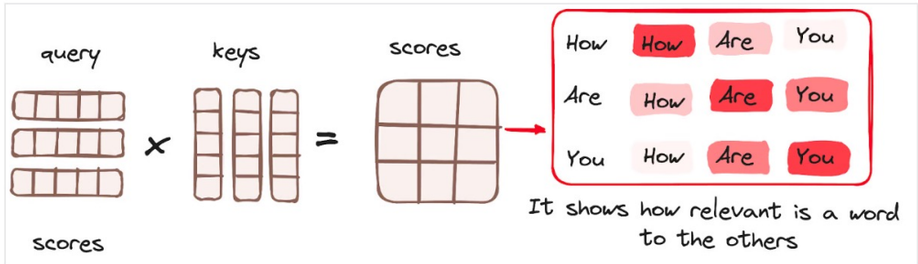
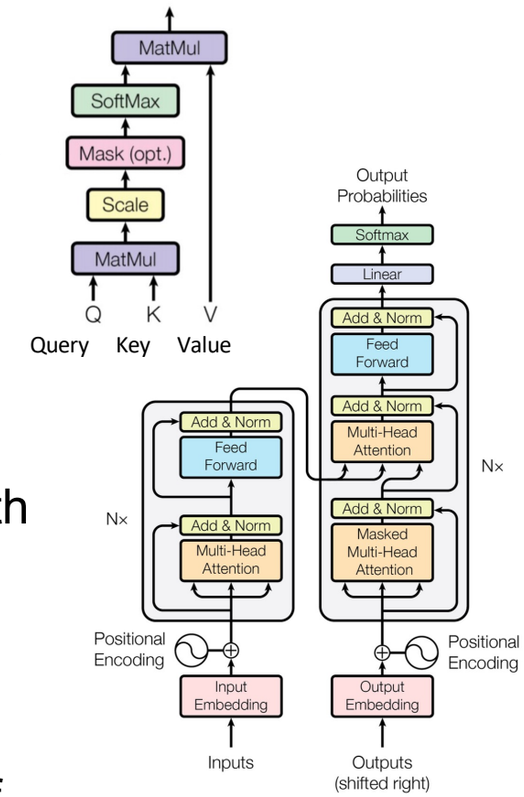


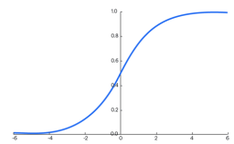
Image by the author. Encoder's workflow. Attention mechanism - Matrix Multiplication.

https://www.datacamp.com/tutorial/how-transformers-work

Main mathematical operations of a transformer

- **Matrix Multiplication** - Used throughout the architecture, particularly in the attention mechanism and feed-forward networks.
- **Attention Mechanism** - The core operation involving:
  - Query, Key, Value transformations (matrix multiplications)
  - Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$ [GPU SFU]
  - Multi-head attention which runs multiple attention operations in parallel
- **Layer Normalization** - Normalizes the outputs of sub-layers using mean and variance statistics
- **Residual Connections** (Skip Connections) - Addition operations that help with gradient flow
- **Position-wise Feed-Forward Networks** - Two linear transformations with a ReLU activation in between
- **Softmax** - Used in the attention mechanism to convert scores to probabilities
- **Positional Encoding** - Often implemented using sine and cosine functions of different frequencies [GPU SFU]
- **Embedding** - Converting tokens to continuous vector representations





$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Softmax is a function that converts a vector of arbitrary real numbers into a probability distribution. All outputs are positive and sum to exactly 1.

Christof Teuscher • teuscher@pdx.edu

How to map transformers on a GPU

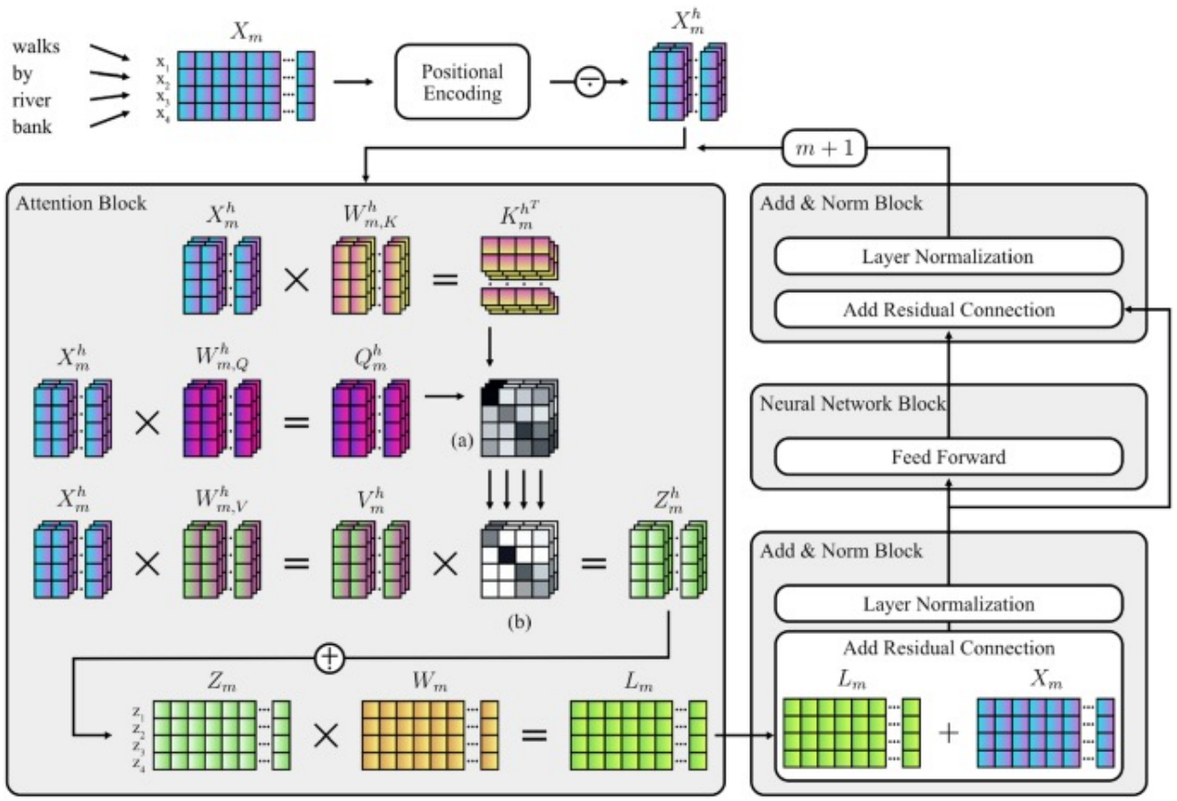


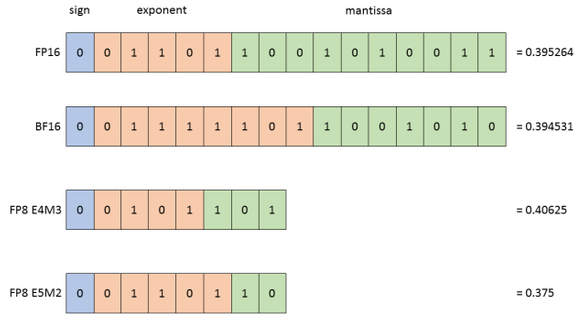
FIGURE 1.
Schematic Diagram of the Attention-Mechanism and Components of the Transformer Architecture. *Note*. The process illustrates the encoding and transformation of the sequence "*walks by river bank*" by components of the transformer architecture (Vaswani et al., 2017). Weight matrices ($W^h_{m,\,K|Q|V}$ and W_m) are randomly initialized and then learned during the training process. In case of causal language models, masking (see Eq. 5) is applied to Z^h_m. (a) = Matrix product of $K^{h^T}_m$ and Q^h_m; (b) Scaling and softmax is applied; n = Input sequence length; d = Model dimensionality, i.e., length of embedding vectors; h = Current attention head; n_h = Number of attention heads; m = Current layer; X_m = Embedding matrix ($dimensionality : n \times d$); X^h_m = Embedding matrix subset ($n \times \frac{d}{n_h}$); $W^h_{m,\,K|Q|V}$ = Key, query, and value weight matrices ($n \times \frac{d}{n_h}$); $K^{h^T}_m$ = Transposed key matrix ($n \times \frac{d}{n_h}$); Q^h_m = Query matrix ($n \times \frac{d}{n_h}$); V^h_m = Value matrix ($n \times \frac{d}{n_h}$); Z_m = Attention matrix ($n \times d$); W_m = Weight matrix ($n \times d$); L_m = Layer output matrix ($n \times d$); $\frac{\cdot}{\cdot}$ = Matrix subdivision; $\frac{+}{\cdot}$ = Matrix concatenation.

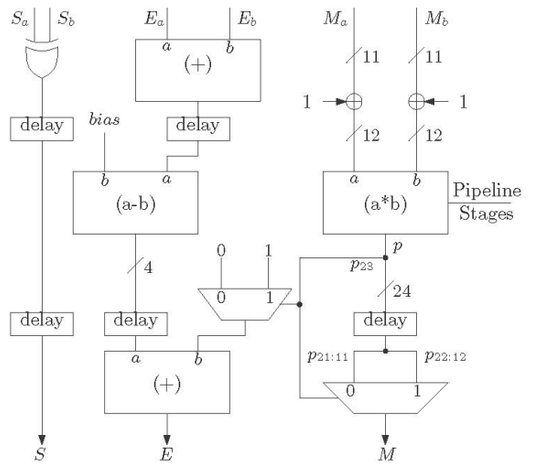
- Q: "What am I looking for?"
- K: "What do I offer?"
- V: "What do I actually contribute if selected?"

Hommel, B.E., Wollang, FJ.M., Kotova, V. et al. **Transformer-Based Deep Neural Language Modeling for Construct-Specific Automatic Item Generation**. Psychometrika 87, 749–772 (2022). https://doi.org/10.1007/s11336-021-09823-9

REMINDER: Why lower precision?

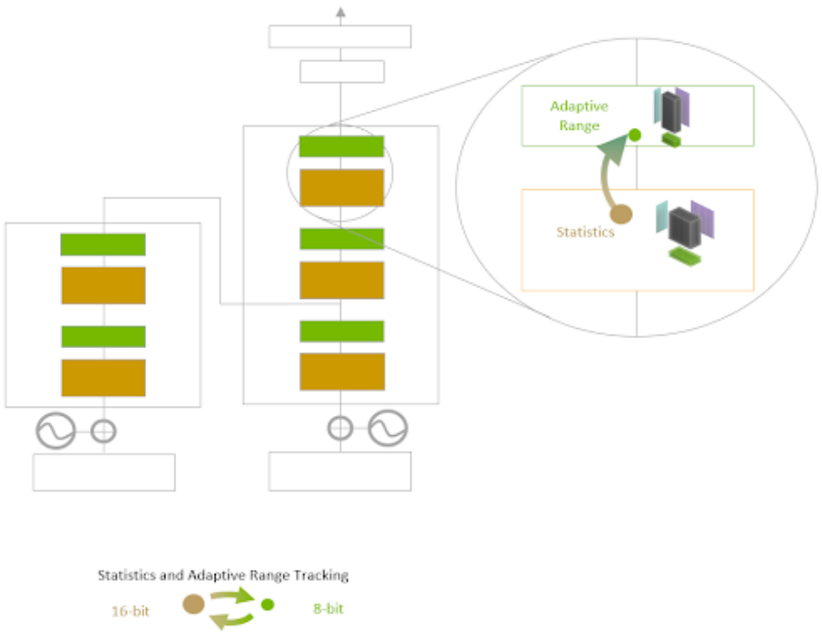
- **Data Transfer Efficiency**: FP8 values are half the size of FP16; FP4 halves again. More data per memory transaction.
- **Cache Efficiency**: Smaller types → more values fit in cache, reducing expensive off-chip memory accesses.
- **Compute Density**: More arithmetic units in the same silicon area → more parallel operations per cycle.
- **Power Efficiency**: Lower precision consumes less power, allowing higher clocks or more ops per watt.
- **FP4 (NEW — 2024+)**: NVIDIA Blackwell (B200) introduces FP4 via 2nd-gen Transformer Engine — 20 PFLOPS sparse; Google's Ironwood (TPU v7) adds native FP8. Microscaling formats (MXFP4/MXFP6) maintain accuracy at 4-bit.

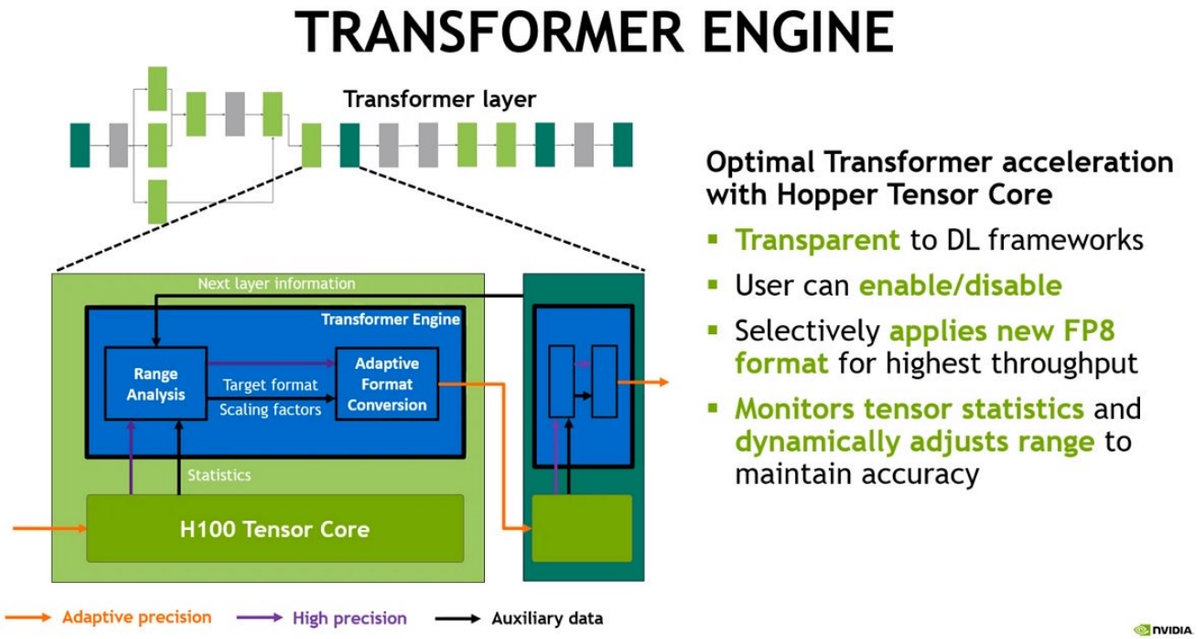




NVIDIA Transformer Engine (1st gen)

Essentially a library that uses the the GPU tensor cores.





Transformer Engine uses per-layer statistical analysis to **dynamically determine the optimal precision (FP16 or FP8) for each layer of a model**, achieving the best performance while preserving model accuracy.

https://www.hardwarezone.com.sg/tech-news-nvidia-h100-gpu-hopper-architecture-building-block-ai-infrastructure-dgx-h100-supercomputer
https://blogs.nvidia.com/blog/h100-transformer-engine
https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/index.html
https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture

Christof Teuscher • teuscher@pdx.edu

NVIDIA Blackwell B200: Key Specifications

Specification	H100 (Hopper)	B200 (Blackwell)
Process	TSMC 4N	**TSMC 4NP**
Transistors	80 B	**208 B (dual-die)**
HBM Capacity	80 GB HBM3	**192 GB HBM3e**
Memory Bandwidth	3.35 TB/s	**8 TB/s**
Tensor Cores	4th gen	**5th gen**
Lowest Precision	FP8	**FP4 (MXFP4)**
Transformer Engine	1st gen	**2nd gen (FP4)**
Peak AI (sparse)	~4 PFLOPS FP8	**20 PFLOPS FP4**
NVLink	4.0 (900 GB/s)	**5.0 (1.8 TB/s)**
TDP	700 W	**1000 W**

What's New in Blackwell

- Dual-die design: Two GB100 dies connected by 10 TB/s interconnect
- FP4 Tensor Cores: 2× throughput vs FP8 for inference; MXFP4 format preserves accuracy via per-block scaling
- 2nd-gen Transformer Engine: Auto mixed-precision between FP4/FP8/BF16 per layer
- NVLink 5: 1.8 TB/s bidirectional — removes PCIe bottleneck for multi-GPU
- GB200 NVL72: 72-GPU rack-scale system; 30× faster LLM inference vs H100

Sources: nvidia.com/blackwell; Wikipedia: Blackwell (microarchitecture)

Journals & Magazines > IEEE Transactions on Circuits... > Early Access

A Neuromorphic Transformer Architecture Enabling Hardware-Friendly Edge Computing

Publisher: IEEE Cite This PDF

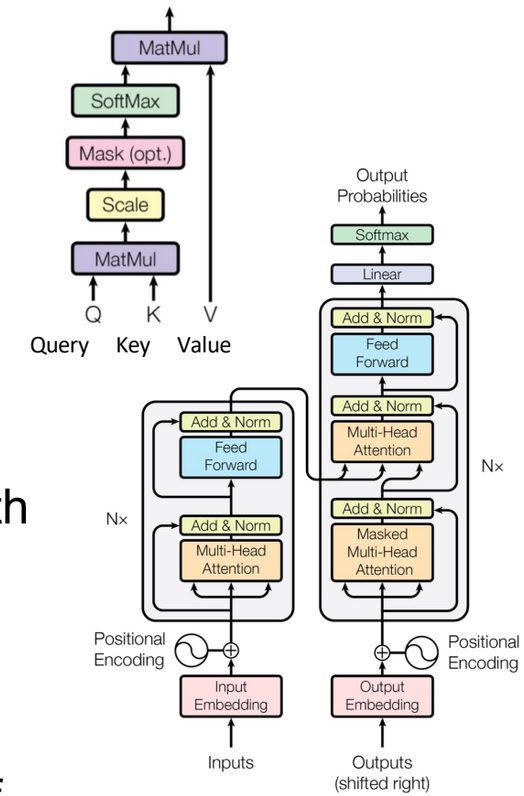
P. J. Zhou ; R. C. Ma ; Y. C. Chen ; Z. T. Liu ; C. Y. Liu ; L. W. Meng **All Authors**

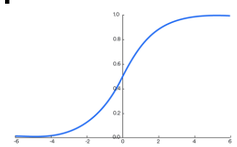
Abstract:
The transformer model has demonstrated significant capabilities in various intelligent tasks, attracting widespread attention in recent years. However, it involves numerous complex operations, including large-bit-width multiplication, division, matrix transposition, and exponentiation. These require substantial storage and computational resources, making it challenging to deploy on edge devices.

https://ieeexplore.ieee.org/abstract/document/10962199

What transformer operations could we improve?

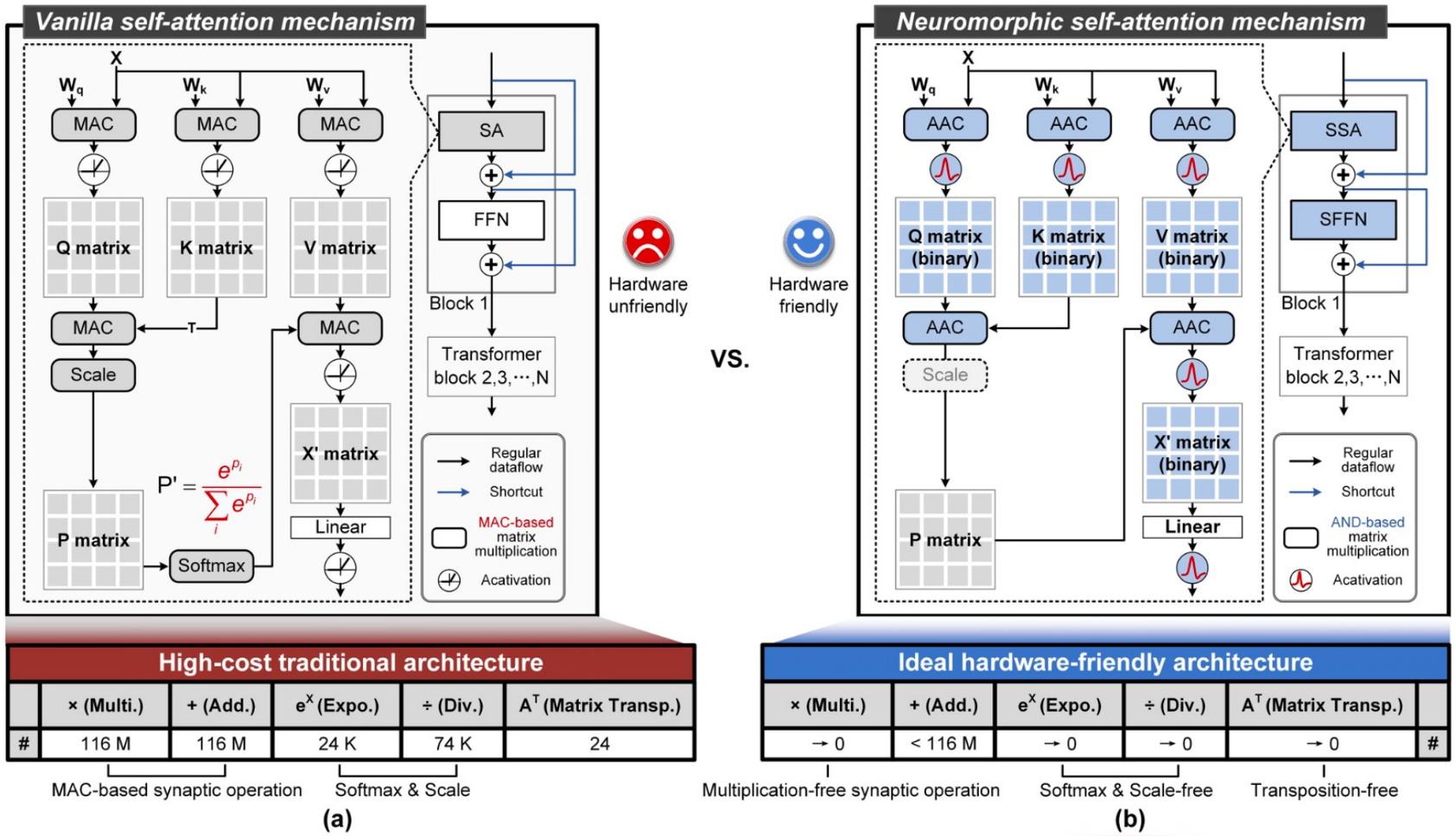
- **Matrix Multiplication** - Used throughout the architecture, particularly in the attention mechanism and feed-forward networks.
- **Attention Mechanism** - The core operation involving:
  - Query, Key, Value transformations (matrix multiplications)
  - Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
  - Multi-head attention which runs multiple attention operations in parallel
- **Layer Normalization** - Normalizes the outputs of sub-layers using mean and variance statistics
- **Residual Connections** (Skip Connections) - Addition operations that help with gradient flow
- **Position-wise Feed-Forward Networks** - Two linear transformations with a ReLU activation in between
- **Softmax** - Used in the attention mechanism to convert scores to probabilities
- **Positional Encoding** - Often implemented using sine and cosine functions of different frequencies
- **Embedding** - Converting tokens to continuous vector representations





$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Neuromorphic self-attention mechanism

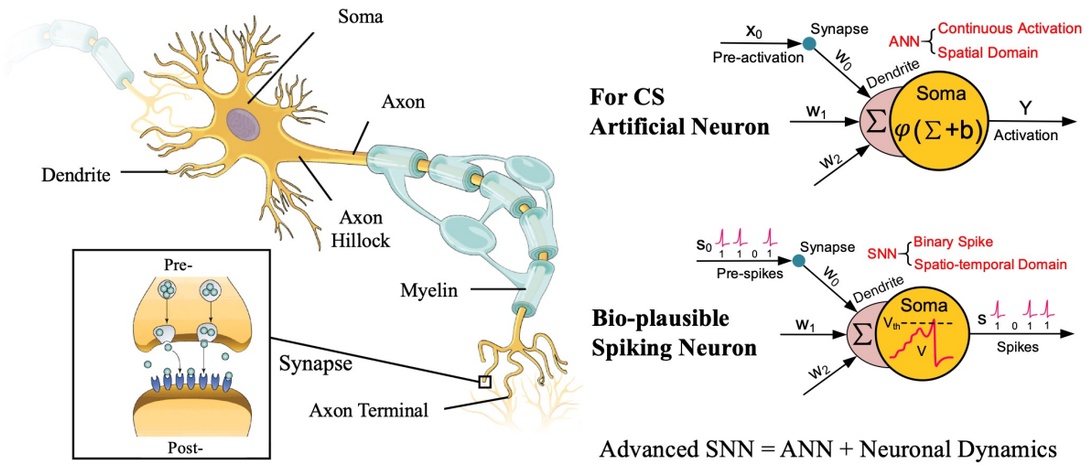


https://ieeexplore.ieee.org/abstract/document/10962199

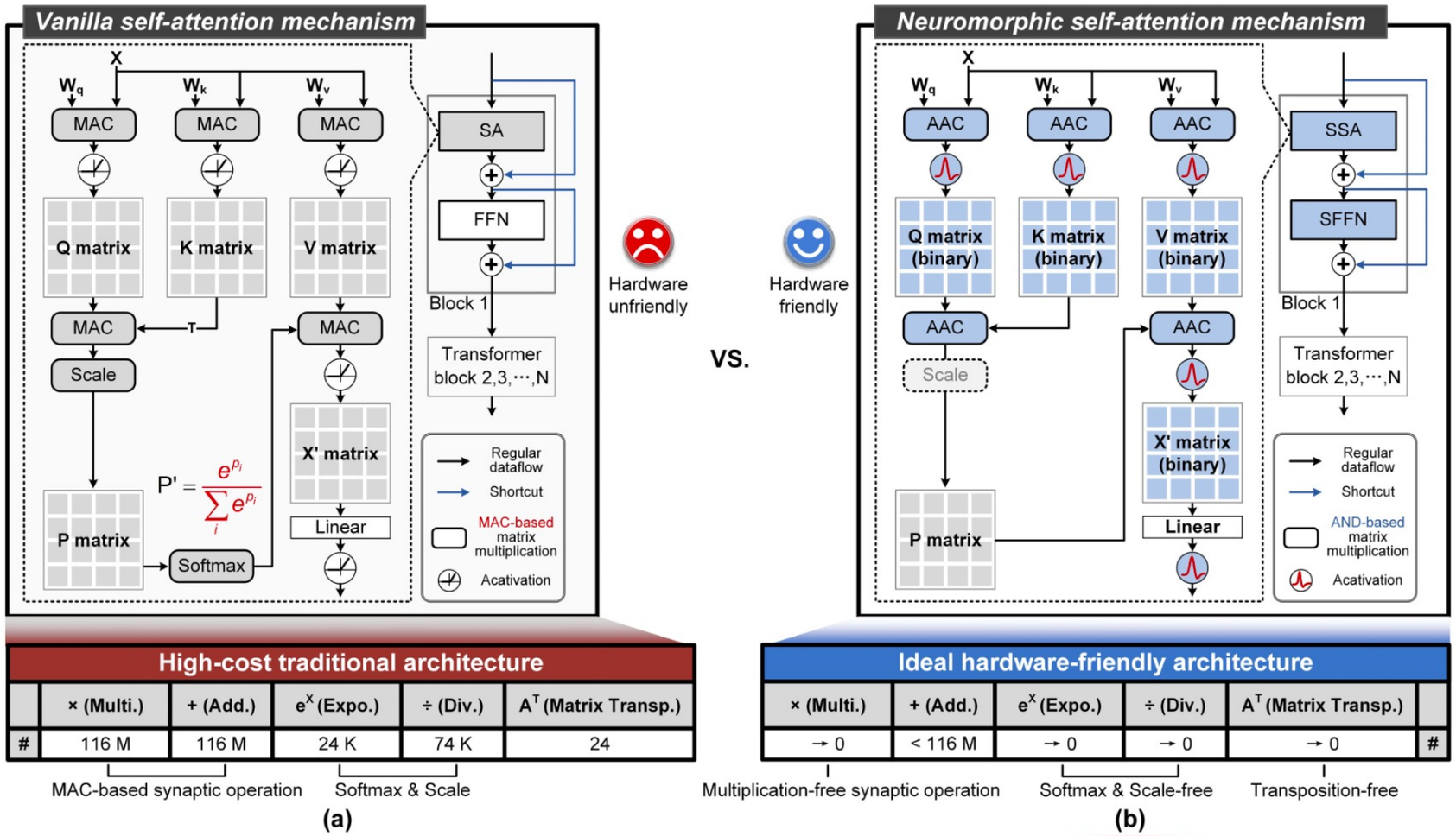
To spike or not to spike

Spiking neural networks are more energy efficient than traditional artificial neural networks primarily because:

- **Event-driven processing**: SNNs only process information when neurons fire, unlike traditional ANNs which compute at every time step regardless of input changes. This significantly reduces power consumption.
- **Binary communication**: SNNs communicate using discrete spikes (binary events) rather than continuous values, requiring less data transfer and simpler hardware.
- **Temporal encoding**: Information in SNNs can be encoded in the timing of spikes, allowing for more efficient data representation compared to the amplitude-based encoding in ANNs.
- **Reduced precision requirements**: SNNs can work with lower precision hardware since they use binary spikes, while ANNs typically need higher precision for gradient calculations.
- **Closer to biological efficiency**: The human brain, which uses spiking neurons, consumes only about 20 watts while performing complex tasks that would require orders of magnitude more power on traditional computing hardware.



Neuromorphic self-attention mechanism



AAC =
AND accumulate

AND accumulate

Traditional:

```
output = (input1 × weight1) + (input2 × weight2) + ... + (inputN × weightN)
```

AND accumulate:

```
output = (input1 AND weight1) + (input2 AND weight2) + ... + (inputN AND weightN)
```

This process essentially counts how many input-weight pairs are both "on" (1) at the same time, which serves as an approximation of multiplication in binary or highly quantized networks.

When input is binary {0,1}, input × weight has only two outcomes:

- If input = 0 → result is 0, regardless of weight
- If input = 1 → result is weight, unchanged

That's exactly what a bitwise AND does: it gates the weight by whether the spike fired. No multiplier circuit needed at all.

Dataflow of transposition calculation

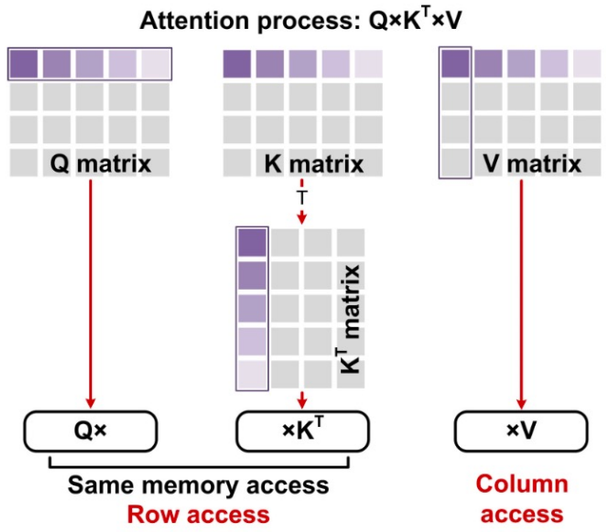


Fig. 4. Matrix multiplication and transposition lead to inconsistent memory access of Q, K, and V.

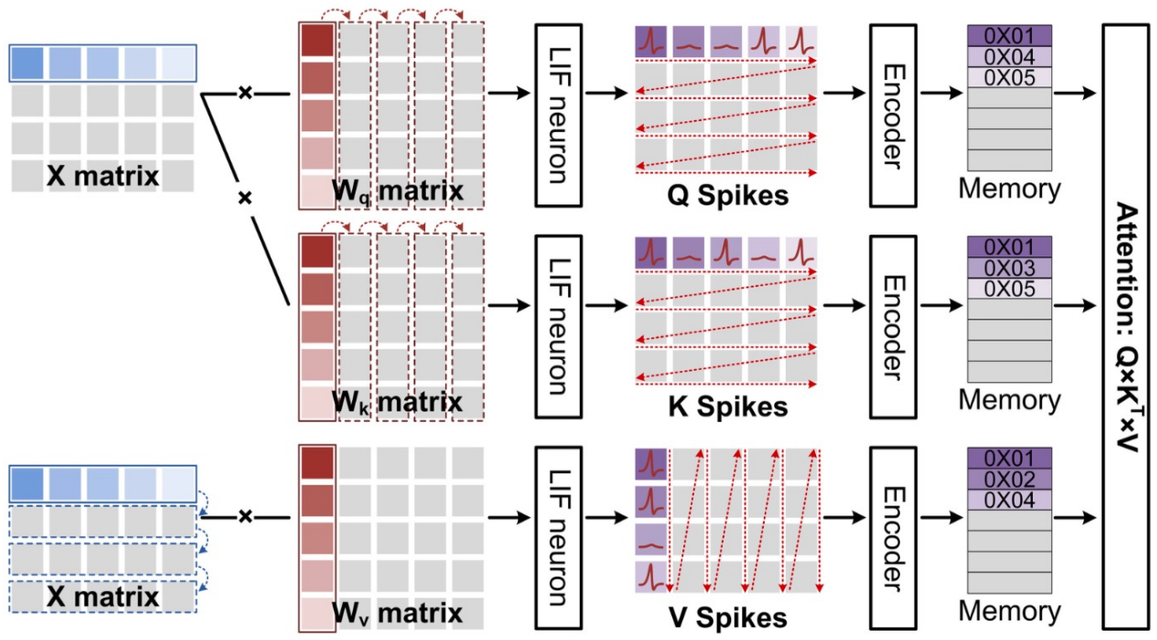
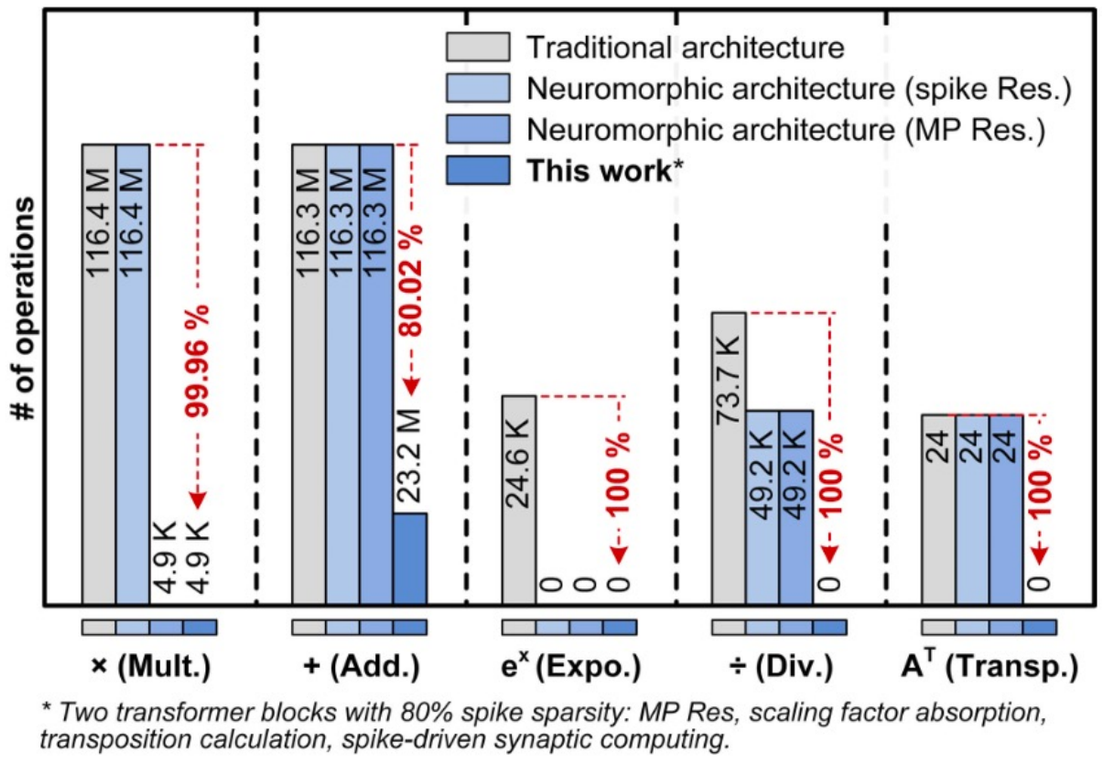


Fig. 5. Data flow design to eliminate memory access problem in the attention mechanism. Q and K are calculated and stored row-by-row, whereas V is calculated and stored column-by-column.

https://ieeexplore.ieee.org/abstract/document/10962199

Resource overhead



* *Two transformer blocks with 80% spike sparsity: MP Res, scaling factor absorption, transposition calculation, spike-driven synaptic computing.*

Fig. 8. Computational resource overhead of different transformer architectures.

Res. = Residuals
MP = membrane potential

https://ieeexplore.ieee.org/abstract/document/10962199

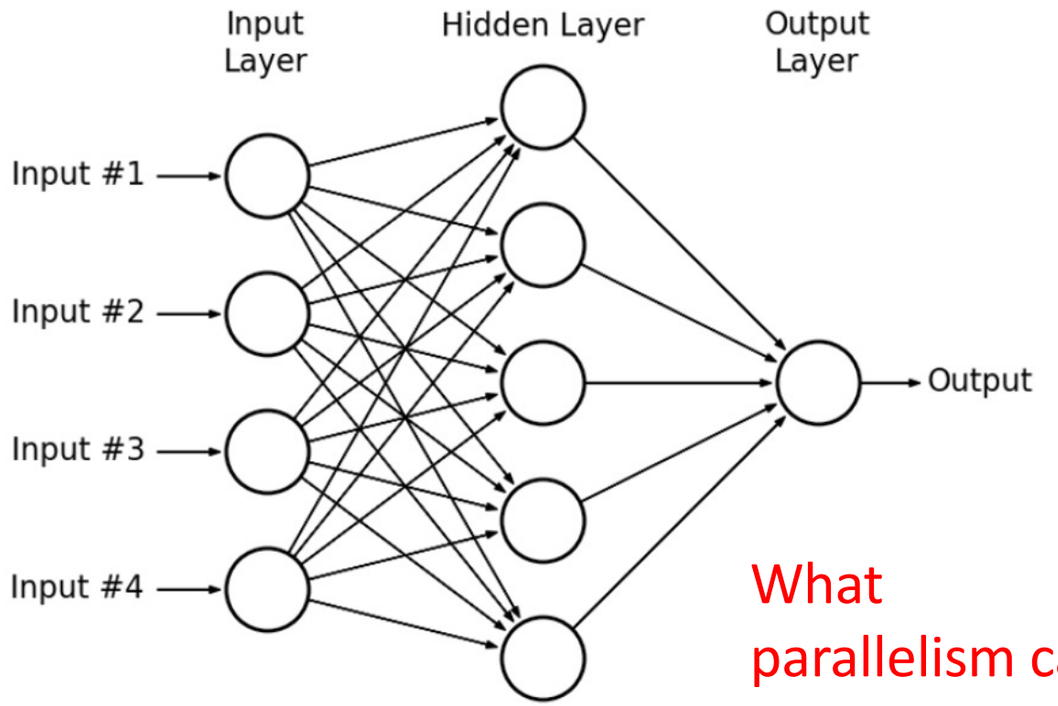
Comparison with state-of-the-art

	JSSC22 [18]	JSSC23 [19]	JSSC24 [48]	TBCAS19 [56]	JSSC24 [43]	NSR24 [57]	**This work**
Architecture	ANN (4×systolic-based attention core)	ANN (1×systolic-based general core)	ANN&SNN (64×FC core)	SNN (4×FC core)	SNN (1024×FC core)	SNN (575×FC core)	**SNN (2×transformer block)**
Technology (nm)	28	65	28	65	28	22	**28**
Die area (mm^2)	6.82	6.40	1.63	3.5	537.98	358.53	**23.28 (all-mem on chip)**[a]
Frequency (MHz)	50-510	600-1100	40-210	210	24-600	300-400	**400**
Power (mW)	12.06-272.7	400-1675	2.91-56.8	19.9	9970	1800	**798.59**[b]
On-chip Memory	336 KB	64 KB	266.5 KB	288 KB	181.865 MB	N. A.	**5.13 MB**
Synaptic precision	INT12	INT8	INT8/10	INT1	INT1/2/4/8	INT1/2/4/8/16	**INT8**
Softmax	Required	Required	-	-	-	-	**Unrequired**
Scale	Required	Required	-	-	-	-	**Unrequired**
Matrix transposition	Required	Required	-	-	-	-	**Unrequired**
Synaptic MAC	Required	Required	Required	Unrequired	Unrequired	Unrequired	**Unrequired**
Accuracy (Cifar-10 dataset)	N. A.	91.5% (CNN)	N. A.	N. A.	93.91% (CNN)	90.17% (CNN)	**94.03 %**[c]
Computing efficiency per core[d] (GSOP/s)	65.25 (MM) 65.25-157.5 (QK^T) 65.25-508.75 (PV) (@510 MHz)	512	N. A.	0.105 (@210 MHz)	20.25 (@600 MHz)	N. A.	**G^f: 1.49-2.93 A^f: 1.10-1.84 F^f: 1.00-1.98**
Energy efficiency[e] (pJ/SOP)	1.047 (MM) 0.77-0.32 (QK^T) 0.77-0.14 (PV) (@510 MHz)	3.33-2 (@1 GHz)	4.16 (@210 MHz)	65 (@210 MHz)	2.444 (@120 MHz)	5.4 (@300-400 MHz)	**G^f: 0.64-0.33 A^f: 4.34-2.79 F^f: 5.71-3.07**

https://ieeexplore.ieee.org/abstract/document/10962199

How to map a neural network on a GPU?

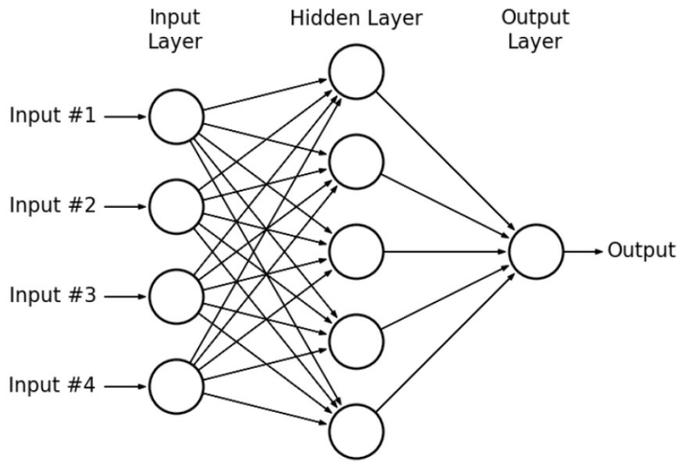
Multi-layer perceptron (MLP)

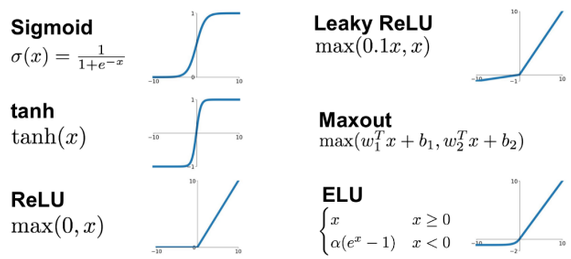


What parallelism can be exploited?

Multi-layer perceptron (MLP)

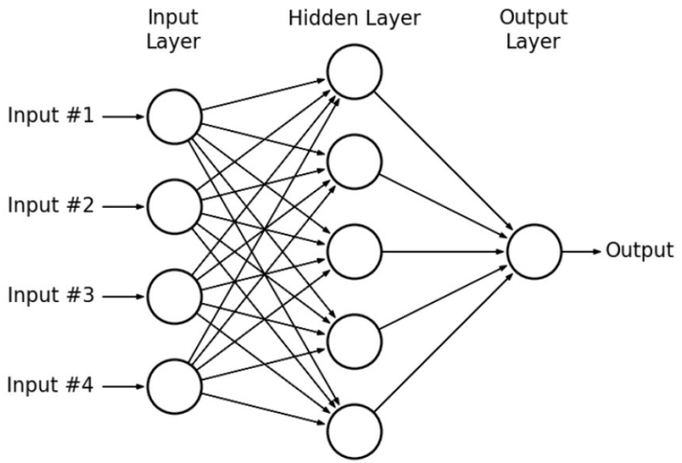
- **Data parallelism**: You can process multiple training examples simultaneously. Since each example is processed independently through the forward pass, you can distribute batches across different computing units.
- **Model parallelism**: The network itself can be partitioned, with different parts of the model assigned to different processors. This works particularly well for very large networks.
- **Layer-wise parallelism**: Within each layer, the computations can be parallelized since neurons in the same layer operate independently.

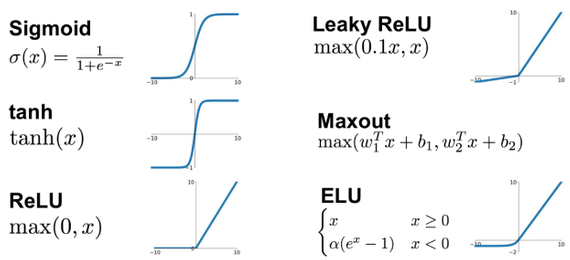




Multi-layer perceptron (MLP)

- **Matrix multiplications**: The core operation in MLPs (Wx + b) is highly parallelizable. Each output neuron's computation is independent.
- **Activation functions**: These are applied element-wise and can be computed in parallel across all neurons.
- **Backpropagation**: During training, gradient computations can be parallelized both within and across layers.



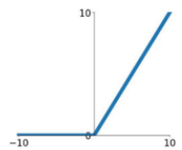


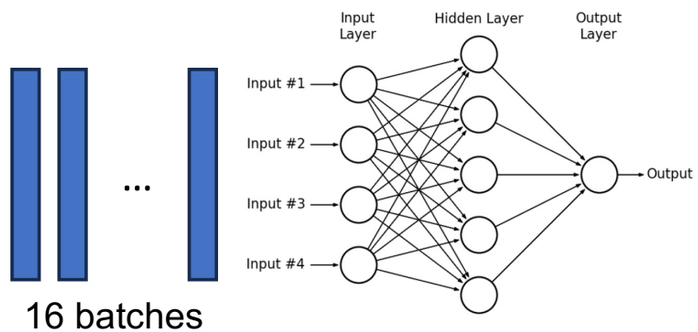
CUDA MLP (1)

```c
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>
#include <time.h>

// Dimensions of our network
#define INPUT_SIZE 4
#define HIDDEN_SIZE 5
#define OUTPUT_SIZE 1
#define BATCH_SIZE 16  // Process 16 samples at once

// Activation function (ReLU)
__device__ float relu(float x) {
    return x > 0 ? x : 0;
}
```

ReLU

$$\max(0, x)$$

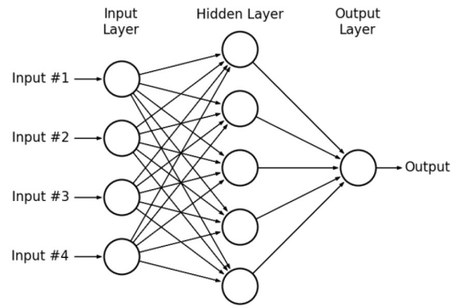




CUDA MLP (2)

```
// Forward pass kernel for first layer (input -> hidden)
__global__ void layer1_forward(float *input, float *weights, float *bias, float *output, int batch_size) {
    int batch_idx = blockIdx.x * blockDim.x + threadIdx.x;
    int neuron_idx = blockIdx.y * blockDim.y + threadIdx.y;

    if (batch_idx < batch_size && neuron_idx < HIDDEN_SIZE) {
        float sum = bias[neuron_idx];
        for (int i = 0; i < INPUT_SIZE; i++) {
            sum += input[batch_idx * INPUT_SIZE + i] * weights[i * HIDDEN_SIZE + neuron_idx];
        }
        output[batch_idx * HIDDEN_SIZE + neuron_idx] = relu(sum);
    }
}

// Forward pass kernel for second layer (hidden -> output)
__global__ void layer2_forward(float *input, float *weights, float *bias, float *output, int batch_size) {
    int batch_idx = blockIdx.x * blockDim.x + threadIdx.x;

    if (batch_idx < batch_size) {
        float sum = bias[0];
        for (int i = 0; i < HIDDEN_SIZE; i++) {
            sum += input[batch_idx * HIDDEN_SIZE + i] * weights[i];
        }
        output[batch_idx] = sum;  // No activation on output layer in this example
    }
}
```

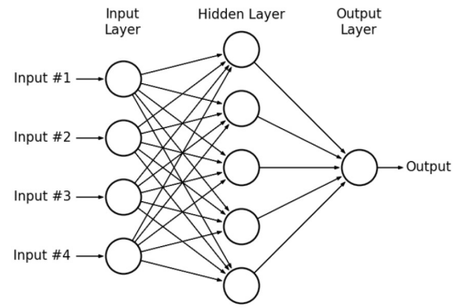
Global thread ID (x coordinate)
Global y coordinate



CUDA MLP (3)

Why separate kernels for each layer?

- **Memory optimization**: Each layer has different memory access patterns and working set sizes. Separate kernels allow you to optimize memory usage specifically for each layer's requirements rather than trying to find a one-size-fits-all approach.
- **Parallelism control**: Different layers may benefit from different thread configurations. For example, the first layer in our implementation handles 4 inputs to 5 neurons, while the output layer handles 5 inputs to 1 neuron - these naturally map to different parallelization strategies.
- **Resource utilization**: GPU resources like shared memory, registers, and thread counts can be tailored specifically to each layer's computational needs, which improves overall efficiency.
- **Kernel fusion limitations**: While fusing operations into a single kernel can reduce launch overhead, there are practical limits to how much computation can be packed into one kernel before it becomes inefficient. Separate kernels help manage this complexity.
- **Synchronization points**: Having distinct kernels creates natural synchronization points between layers, ensuring all computations from one layer are complete before the next layer begins processing.
- **Debugging and profiling**: With separate kernels, it's easier to identify performance bottlenecks or errors in specific parts of the network.
- **Specialized optimizations**: Different mathematical operations may benefit from specialized implementations - convolutions, fully-connected layers, and activation functions all have different optimal implementations on GPUs.

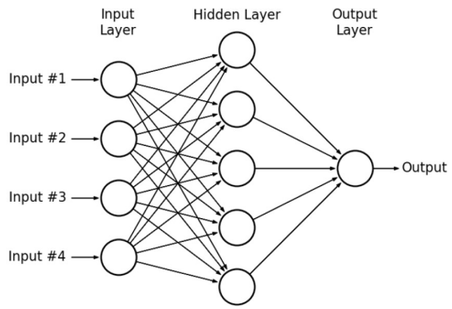


CUDA MLP (4)

How much should be in a kernel?

The optimal kernel size involves balancing several trade-offs:

- **Kernel launch overhead**: Each kernel launch incurs some overhead. Too many small kernels can result in performance degradation due to this overhead.
- **Register pressure**: Larger kernels with more operations typically require more registers per thread, which can reduce occupancy (the number of threads that can run simultaneously on the GPU).
- **Instruction cache**: GPUs have limited instruction cache. Very large kernels might not fit well in the cache, causing additional latency.
- **Memory bandwidth utilization**: Kernels that load data, perform a small operation, and then store results can be memory-bound. Combining multiple operations in one kernel can improve arithmetic intensity and make better use of compute resources.
- **Thread divergence**: If your kernel contains many conditional branches that cause different threads to follow different execution paths, performance will suffer.

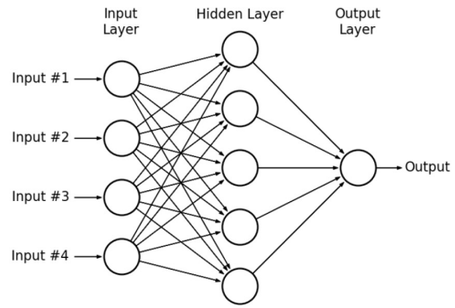


As a general guideline:

- **Good candidates for fusion**: Operations that work on the same data and have similar parallelization patterns (like a matrix multiplication followed by an element-wise activation function).
- **Better kept separate**: Operations with fundamentally different access patterns or parallelization strategies.

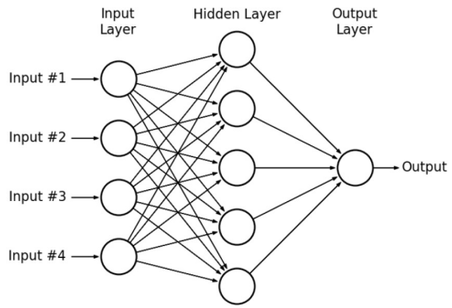
CUDA MLP (5)

```
// Initialize weights with random values between -0.5 and 0.5
void initialize_weights(float *h_weights, int size) {
    for (int i = 0; i < size; i++) {
        h_weights[i] = ((float)rand() / RAND_MAX) - 0.5f;
    }
}

int main() {
    // Seed random number generator
    srand(time(NULL));

    // Host memory allocation
    float h_input[BATCH_SIZE * INPUT_SIZE];
    float h_hidden[BATCH_SIZE * HIDDEN_SIZE];
    float h_output[BATCH_SIZE * OUTPUT_SIZE];
    float h_weights1[INPUT_SIZE * HIDDEN_SIZE];
    float h_bias1[HIDDEN_SIZE];
    float h_weights2[HIDDEN_SIZE * OUTPUT_SIZE];
    float h_bias2[OUTPUT_SIZE];

    // Initialize weights and biases
    initialize_weights(h_weights1, INPUT_SIZE * HIDDEN_SIZE);
    initialize_weights(h_bias1, HIDDEN_SIZE);
    initialize_weights(h_weights2, HIDDEN_SIZE * OUTPUT_SIZE);
    initialize_weights(h_bias2, OUTPUT_SIZE);

    // Initialize some example input data
    for (int i = 0; i < BATCH_SIZE * INPUT_SIZE; i++) {
        h_input[i] = ((float)rand() / RAND_MAX);
    }
```



CUDA MLP (6)

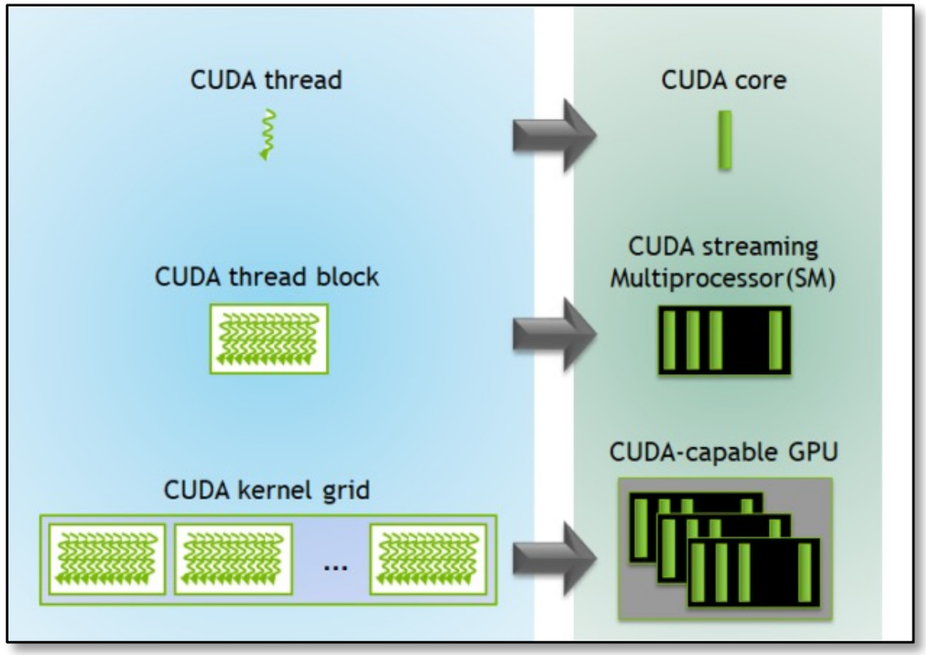
```
// Host memory allocation
float h_input[BATCH_SIZE * INPUT_SIZE];
float h_hidden[BATCH_SIZE * HIDDEN_SIZE];
float h_output[BATCH_SIZE * OUTPUT_SIZE];
float h_weights1[INPUT_SIZE * HIDDEN_SIZE];
float h_bias1[HIDDEN_SIZE];
float h_weights2[HIDDEN_SIZE * OUTPUT_SIZE];
float h_bias2[OUTPUT_SIZE];
```

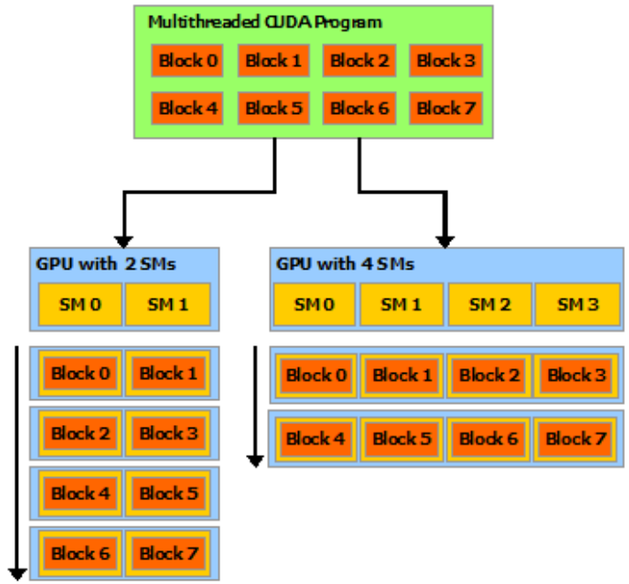


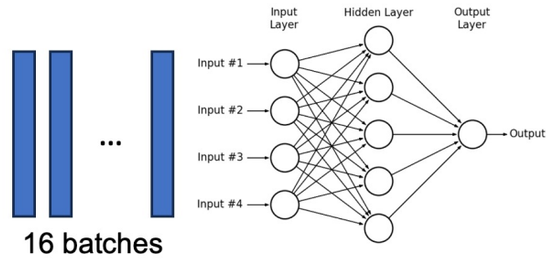

```
// Device memory allocation
float *d_input, *d_hidden, *d_output;
float *d_weights1, *d_bias1, *d_weights2, *d_bias2;

cudaMalloc(&d_input, BATCH_SIZE * INPUT_SIZE * sizeof(float));
cudaMalloc(&d_hidden, BATCH_SIZE * HIDDEN_SIZE * sizeof(float));
cudaMalloc(&d_output, BATCH_SIZE * OUTPUT_SIZE * sizeof(float));
cudaMalloc(&d_weights1, INPUT_SIZE * HIDDEN_SIZE * sizeof(float));
cudaMalloc(&d_bias1, HIDDEN_SIZE * sizeof(float));
cudaMalloc(&d_weights2, HIDDEN_SIZE * OUTPUT_SIZE * sizeof(float));
cudaMalloc(&d_bias2, OUTPUT_SIZE * sizeof(float));

// Copy data from host to device
cudaMemcpy(d_input, h_input, BATCH_SIZE * INPUT_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_weights1, h_weights1, INPUT_SIZE * HIDDEN_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_bias1, h_bias1, HIDDEN_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_weights2, h_weights2, HIDDEN_SIZE * OUTPUT_SIZE * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_bias2, h_bias2, OUTPUT_SIZE * sizeof(float), cudaMemcpyHostToDevice);
```

CUDA MLP (7)

How will we use threads for this MLP?







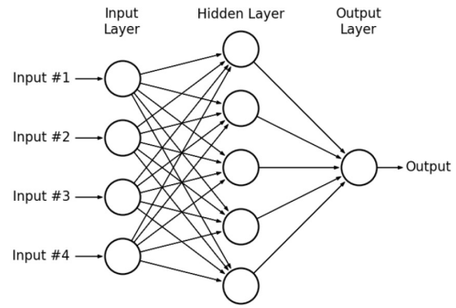
A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program is partitioned into blocks of threads that execute independently from each other**.

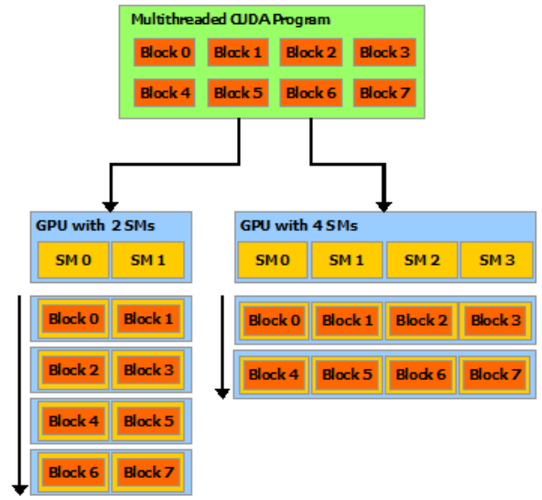
CUDA MLP (8)

```
// Set up execution configuration for first layer
dim3 threadsPerBlock1(8, 5);  // 8 batches, all 5 hidden neurons
dim3 blocksPerGrid1((BATCH_SIZE + threadsPerBlock1.x - 1) / threadsPerBlock1.x,
                    (HIDDEN_SIZE + threadsPerBlock1.y - 1) / threadsPerBlock1.y);

// Set up execution configuration for second layer
dim3 threadsPerBlock2(16, 1);  // 16 batches
dim3 blocksPerGrid2((BATCH_SIZE + threadsPerBlock2.x - 1) / threadsPerBlock2.x, 1);

// Launch kernels
layer1_forward<<<blocksPerGrid1, threadsPerBlock1>>>(d_input, d_weights1, d_bias1, d_hidden, BATCH_SIZE);
layer2_forward<<<blocksPerGrid2, threadsPerBlock2>>>(d_hidden, d_weights2, d_bias2, d_output, BATCH_SIZE);

// Check for errors
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) {
    printf("CUDA Error: %s\n", cudaGetErrorString(err));
    return -1;
}

// Copy results back to host
cudaMemcpy(h_output, d_output, BATCH_SIZE * OUTPUT_SIZE * sizeof(float), cudaMemcpyDeviceToHost);

// Print some outputs
printf("Sample outputs from MLP:\n");
for (int i = 0; i < 5; i++) {  // Print first 5 results
    printf("Sample %d: %.6f\n", i, h_output[i]);
}
```

Why?

`kernel<<M,T>>`
The kernel launches with a grid of M thread blocks.
Each thread block has T parallel threads.





A GPU is built around an **array of Streaming Multiprocessors (SMs). A multithreaded program is partitioned into blocks of threads that execute independently from each other**.

CUDA MLP (9)

```
    // Free GPU memory
    cudaFree(d_input);
    cudaFree(d_hidden);
    cudaFree(d_output);
    cudaFree(d_weights1);
    cudaFree(d_bias1);
    cudaFree(d_weights2);
    cudaFree(d_bias2);
```